武漢大學

本科毕业论文（设计）

嵌入式系统中高效二进制日志管理机制的研究与设计

姓　　名：　　邓泽宇

学　　号：　　2022302111098

专　　业：　　软件工程

学　　院：　　计算机学院

指导教师：　　武小平

二〇二六年三月

原创性声明

本人郑重声明：所呈交的论文（设计），是本人在指导教师的指导下，严格按照学校和学院有关规定完成的。除文中已经标明引用的内容外，本论文（设计）不包含任何其他个人或集体已发表及撰写的研究成果。对本论文（设计）做出贡献的个人和集体，均已在文中以明确方式标明。本人承诺在论文（设计）工作过程中没有伪造数据等行为。若在本论文（设计）中有侵犯任何方面知识产权的行为，由本人承担相应的法律责任。

作者签名：_____　　指导教师签名：_____

日　期：_____

版权使用授权书

本人完全了解武汉大学有权保留并向有关部门或机构送交本论文（设计）的复印件和电子版，允许本论文（设计）被查阅和借阅。本人授权武汉大学将本论文的全部或部分内容编入有关数据库进行检索和传播，可以采用影印、缩印或扫描等复制手段保存和汇编本论文（设计）。

作者签名：_____　　指导教师签名：_____

日　期：_____

摘　要

嵌入式与边缘设备中的日志系统既承担故障诊断、运行观测与行为追踪等任务，又受到计算能力、存储容量和并发访问条件的严格约束。现有文本日志方案虽然具备良好的可读性与兼容性，但在高频写入场景下普遍存在格式化开销高、冗余存储多以及共享元数据重复解析的问题；当多个进程或节点共同依赖同一份格式信息时，还容易引入初始化竞争与一致性风险。针对上述矛盾，本文设计并实现了基于共享事件格式的嵌入式二进制日志系统 Optbinlog。

本文围绕“机制设计—理论建模—实验验证”的主线展开研究。首先，针对日志热路径冗长和共享元数据并发初始化不稳定的问题，提出由共享 schema、偏移访问、自描述帧和记录级校验构成的系统方案；其次，建立由日志条数 N 与并发设备数 D 共同驱动的时间、空间与吞吐模型，用于刻画不同模式下固定项、边际项与竞争项之间的关系；最后，在统一统计口径和严格语义对齐条件下，对单机、本地多设备、真实多设备代理场景以及日志量扩展场景进行系统验证。

实验结果表明，Optbinlog 相较文本路径能够稳定降低时间与空间开销；在严格公平口径下，Optbinlog 对 syslog/ulog 表现出稳定的综合优势，而面对 nanolog/zephyr 等极短热路径方案时，单位记录常数项仍有进一步压缩空间；hilog 则在不同层级场景中呈现“空间持续占优、时间与吞吐易受并发路径影响”的过渡特征。空间交叉点实验进一步表明，Optbinlog 在对标 syslog/ulog 时可较早释放空间收益，在对标 hilog 时也具备可观测的反超区间；初始化竞争实验则验证了共享格式文件在多进程与节点级并发场景下的单创建者复用语义。理论推导与实测趋势总体一致，说明本文提出的共享事件格式机制在强调跨进程一致性、结构化表达与长期在线部署的嵌入式场景中具有较好的工程适用性。

关键词：嵌入式日志；二进制编码；共享内存；并发初始化；性能建模

ABSTRACT

Embedded and edge logging systems must provide observability under tight constraints on computation, storage, and concurrent access. Although text-based logging is easy to read and widely compatible, it incurs high formatting overhead, substantial storage redundancy, and repeated metadata parsing in high-frequency workloads. When multiple processes or nodes share the same event description, initialization races and consistency risks further complicate deployment. To address these issues, this thesis designs and implements Optbinlog, an embedded binary logging system built on shared event schemas.

The study follows a mechanism-model-evidence workflow. First, Optbinlog is designed with shared schemas, offset-based access, self-describing frames, and per-record integrity checks so that schema parsing is moved out of the steady-state hot path while cross-process consistency is preserved. Second, analytical models are developed to characterize time, space, and throughput as functions of log count N and concurrent device count D, with explicit separation of fixed costs, marginal costs, and contention terms. Third, the design is evaluated under unified statistical rules and strict semantic alignment across single-node, local multi-device, real multi-node proxy, and log-volume scaling scenarios.

The results show that Optbinlog consistently improves over text-based paths in both time and space. Under strict fairness, it delivers stable overall advantages against syslog/ulog, still leaves room for reducing per-record constant overhead when compared with ultra-short-path peers such as nanolog/zephyr, and shows a mixed profile against hilog where space remains favorable while time/throughput become scenario-sensitive under node-level contention. Space-crossover experiments further indicate that Optbinlog can release storage benefits early when compared with syslog/ulog, and can also surpass hilog beyond a moderate log volume. Initialization-race experiments validate the single-creator reuse semantics of the shared schema file in both multi-process and node-level concurrent settings. Overall, the empirical trends agree well with the theoretical analysis, indicating that the proposed shared-schema mechanism is well suited to embedded deployments that value cross-process consistency, structured semantics, and long-running online operation.

Keywords: embedded logging; binary encoding; shared metadata; concurrency; performance modeling

目录

摘要 I

ABSTRACT II

1 绪论 1

1.1 研究背景 1

1.2 研究目标与问题定义 1

1.3 主要贡献 2

1.4 研究方法与技术路线 2

1.5 创新点与研究价值 2

2 国内外研究现状与问题归纳 3

2.1 研究对象与评估维度 3

2.2 国际代表系统：机制与实现路径（代码级） 3

2.3 国内代表系统：机制与实现路径（代码级） 5

2.4 Optbinlog 与代表性 peer 系统的对比 5

2.5 典型优化代码入口（可追溯） 9

2.6 现有方案的共性不足 10

2.7 Optbinlog 的针对性设计 10

2.8 复现边界与结论适用范围 11

3 Optbinlog 系统设计与工程实现 11

3.1 设计目标 11

3.2 事件格式与元数据结构 12

3.3 写入/读取路径 13

3.4 关键优化路径与收敛 15

3.5 工程场景适配性 15

3.6 读取与复核能力设计 16

3.7 断电场景下的截断恢复机制 16

4 理论模型与数学推导 17

4.1 符号定义 17

4.2 从算法与数据结构到单条代价 18

4.3 空间随 N 的详细推导 21
4.4 时间随 N 的详细推导 22
4.5 多设备 D 的详细推导 24
4.6 模式间理论排序与可检验预测 26
4.7 与第 5 章指标的映射关系 27
4.8 本章小结 28

5 实验设计、结果与理论一致性分析 28
5.1 实验设计与评估口径 28
5.2 初始化竞争正确性（前置验证） 30
5.3 单机场景：严格对齐下的基线关系 31
5.4 本地多设备：竞争项放大后的趋势 33
5.5 真实多设备：共享格式文件下的节点级结果 37
5.6 日志量扩展：空间交叉点实验 43
5.7 理论—实证对照 45
5.8 原因分析与消融佐证 45
5.9 证据链闭环与外推边界 46
5.10 机制解释与应用启示 46

6 结论与未来展望 47
6.1 研究结论 47
6.2 当前局限 47
6.3 未来展望 48

参考文献 49

致谢 51

附录 52

1 绪论

1.1 研究背景

随着嵌入式设备、边缘网关和轻量控制节点持续承担更复杂的在线业务，日志已经不再只是离线排障时的附属产物，而是系统运行观测、故障定位和行为复核的重要基础设施。[4,12,14,20] 与通用服务器环境相比，这类设备通常同时受到 CPU 性能、内存容量、持久化空间和实时响应要求的约束，因此日志系统不仅要“能记录”，还要尽可能缩短热路径、控制空间增量，并在长期运行中保持稳定的并发行为。

传统文本日志在可读性、工具兼容性和运维生态方面具有明显优势，但在高频写入场景下，字符串格式化、冗余字符存储和重复解析会持续推高时间与空间成本。对于多进程协作或多节点部署场景，如果各执行单元分别解析格式信息、独立维护元数据，不仅启动成本难以收敛，还可能在共享资源首次创建时引入初始化竞争和状态不一致问题。[1,3,7,8]

围绕降低日志开销这一目标，现有研究已经形成了多条技术路径，例如编译期模板提取、无锁缓冲、异步批刷和离线恢复文本等。然而，这些方案往往分别强调极短热路径、高吞吐追踪或后处理能力，对于“共享格式元数据、跨进程一致性与结构化语义可恢复”这一组工程需求，尚缺少统一的机制设计与系统化验证。基于此，本文将研究重点放在面向嵌入式部署的结构化二进制编码与共享元数据机制上，并尝试在统一的理论—实证框架中回答其性能边界与适用条件。

1.2 研究目标与问题定义

基于上述背景，本文希望解决的并不是单一的“写得更快”问题，而是在语义完整性、并发一致性和综合开销之间寻找可解释、可验证的工程折中。围绕这一目标，本文聚焦以下四个核心问题：

1. 现有代表性日志系统在热路径优化、并发组织与空间控制方面分别采用了哪些机制，其适用边界是什么。
2. Optbinlog 的共享元数据与二进制编码路径能否在保持语义完整性的前提下弥补现有方案缺口。
3. 在日志条数 N 与并发设备（节点）数 D 变化时，时间、空间与吞吐应呈现怎样的理论趋势与交叉关系。

4. 理论预测与实验观测在何处一致、何处偏离，偏离由哪些实现常数项与系统路径因素导致。

1.3 主要贡献

围绕上述问题，本文的主要工作与贡献可归纳为以下四点：

1. 设计并实现面向嵌入式多进程场景的 Optbinlog 系统，形成由共享 schema、偏移访问、自描述帧和初始化状态机组成的完整实现方案。
2. 建立由 N 与 D 驱动的统一分析模型，对时间、空间和吞吐的变化趋势、交叉点与竞争放大效应给出可检验表达。
3. 在统一统计规则和严格语义对齐条件下，完成单机、本地多设备、真实多设备代理场景、空间交叉点扫描与初始化竞争验证，形成闭环证据链。
4. 基于实验结果识别出不同 peer 的分组特征与适用边界，说明 Optbinlog 在 syslog/ulog 场景中的稳定优势、在 hilog 场景中的过渡特征，以及在 nanolog/zephyr 场景下仍需继续优化的方向。

1.4 研究方法与技术路线

本文采用“问题驱动、机制设计先行、分层验证跟进”的研究路径。与单纯围绕实验结果组织论文不同，本文更强调先解释代价来源，再讨论实验排序。具体而言，研究方法可概括为三条主线：

1. **机制分析主线**：以算法与数据结构为起点，先解释代价来源，再进行性能比较。
2. **公平对照主线**：在保留工程近似层的同时，引入严格语义对齐层，避免将语义差异误判为实现差异。
3. **理论—实证闭环主线**：统一统计口径与复现实验脚本，按“理论推导 → 单机 → 多设备 → 真实多设备”逐层验证。

1.5 创新点与研究价值

在工程论文写作中，“创新”不仅体现在提出新机制，也体现在能否建立与实验一致的解释框架。本文的创新点与研究价值主要体现在以下几个方面：

1. 提出面向用户态的“共享 schema + 偏移访问”二进制日志机制，缓解多进程初始化一致性与重复解析问题。

2. 给出 N、D 驱动下的时间/空间/吞吐统一推导框架，并提供可直接检验的交叉点与峰值点表达。
3. 建立“工程近似层 + 严格公平层”的双层证据体系，提高结论的可解释性与可迁移性。
4. 在真实多设备节点级并发场景下完成共享格式文件与初始化竞争验证，形成可复现的工程化实验流程。

---

2 国内外研究现状与问题归纳

2.1 研究对象与评估维度

本章的目标不是简单罗列相关系统，而是回答两个与本文设计直接相关的问题：现有日志系统主要在什么层面降低开销，以及这些机制为什么不足以直接覆盖本文关注的共享元数据场景。为此，本文基于官方文档、公开论文与代码仓库开展调研，重点关注“热路径优化方式”和“可复现实验所能对应的实现依据”。[1,3-4,7-8,12,14] 为保证后续比较具有可比性，所有系统统一从四个维度展开：数据结构组织、关键算法路径、并发行为特征与工程部署边界。

调研对象如下：1. Linux ftrace / kernel ring buffer。2. LTTng。3. Zephyr logging subsystem。4. systemd-journald。5. NanoLog（USENIX ATC 2018）。6. RT-Thread ulog。7. OpenHarmony HiLog Lite。

2.2 国际代表系统：机制与实现路径（代码级）

2.2.1 Linux ftrace（内核追踪）

代码与文档显示其核心是 per-CPU lockless ring buffer。在 ring_buffer.c 中，写入路径围绕“预留 → 填充 → 提交”组织，关键是把竞争局部化到 CPU 本地缓冲，降低全局锁竞争。[1-2,15]

1. 数据结构：每 CPU buffer + page 链式组织，维护 head/commit/tail。
2. 算法逻辑：优先本地写入、批量可见性提交，读写阶段解耦。
3. 场景定位：内核 tracing、高并发诊断；功能强但接入复杂度高于应用层日志库。

2.2.2 LTTng（Linux Tracing Toolkit Next Generation）

LTTng 采用 tracepoint instrumentation + lockless buffer 管线，强调高吞吐追踪和离线分析。[3]

1. 数据结构：channel/sub-buffer 等追踪缓冲结构。
2. 算法逻辑：将事件采集与后处理分层，追踪点开销可控。
3. 场景定位：系统级性能分析与可观测平台，部署和运维要求较高。

2.2.3 Zephyr logging（RTOS 场景）

在 Zephyr Kconfig.mode 与 log_msg.c 可见，关键机制是 deferred logging。[4-6]

1. 数据结构：log_msg 对象 + 消息池（MPSC packet buffer）。
2. 算法逻辑：前台线程只做封包入队，后台线程统一输出；支持 overflow/drop 策略。
3. 场景定位：RTOS 中实时任务优先，要求前台日志路径尽量短、可控。

2.2.4 systemd-journald

其 JOURNAL_FILE_FORMAT 与 journald-server 路径强调二进制对象与索引。[7,16]

1. 数据结构：header + objects + 哈希/索引。
2. 算法逻辑：写入与检索并重，保证一致性与查询能力。
3. 场景定位：通用 Linux 系统服务日志；对轻量嵌入式而言依赖偏重。

2.2.5 NanoLog（研究型高性能日志）

从 NanoLogCpp17.h、RuntimeLogger.cc 可见，NanoLog 核心是“编译期提取格式 + 运行期仅记录动态参数”。[8-10]

1. 数据结构：staging buffer、runtime logger 队列/缓冲。
2. 算法逻辑：前台路径尽量只做参数打包，压缩与格式恢复转移到后台/离线。
3. 场景定位：超高频日志、允许离线恢复文本的性能敏感场景。

2.3 国内代表系统：机制与实现路径（代码级）

2.3.1 RT-Thread ulog

ulog.c 提供异步输出和多 backend 抽象，思路是“前台快写缓冲，后台慢速输出”。[12-13]

1. 数据结构：ringbuffer/rbb、backend 列表。
2. 算法逻辑：异步线程统一调度输出，减轻业务线程阻塞。
3. 场景定位：RT-Thread 设备侧调试与在线日志。

2.3.2 OpenHarmony HiLog Lite

HiLog Lite 在格式能力上做约束（减少复杂格式），换取更低资源开销。[14]

1. 数据结构：轻量日志缓冲和简化格式存储。
2. 算法逻辑：限制格式复杂度，降低格式化路径成本。
3. 场景定位：资源受限设备的系统日志基础设施。

2.4 Optbinlog 与代表性 peer 系统的对比

2.4.1 代表性 peer 对比与场景分工

需要说明的是，前述调研覆盖了更广的国内外系统范围，而第 5 章的定量复现只选择其中具有代表性的六类 peer：syslog/ftrace/nanolog/zephyr/ulog/hilog。选择依据是机制覆盖度、用户态可比性与实验可复现性。[1,4,8,12-15] LTTng 与 systemd-journald 在本文中承担机制参照作用，不直接进入第 5 章统一口径下的数值对比。

表 2.1 Optbinlog 与六类 peer 系统对比

系统	核心机制	典型使用场景	主要优势	主要代价	与 Optbinlog 的关系
Linux ftrace	内核 tracing ring buffer	内核/驱动诊断、时序追踪	内核路径短、追踪能力强	接入复杂、与业务日志模型存在语义差异	Optbinlog 适合业务日志结构化记录，Linux ftrace 更适合底层追踪

（续表 2.1）

系统	核心机制	典型使用场景	主要优势	主要代价	与 Optbinlog 的关系
Zephyr logging	deferred 入队 + 后台批刷	RTOS 实时任务日志	前台阻塞小	后台线程调度与队列竞争	工程复现中以 native-like deferred 路径逼近其原生机制；Optbinlog 不依赖后台线程，行为更确定
syslog	文本日志 + 系统日志接口路径	通用 Linux 服务日志	生态成熟、接入通用	文本格式化与系统路径开销较高	Optbinlog 在高频写入场景更强调紧凑编码与可控开销
NanoLog	固定字段二进制直写	超高频、可离线恢复文本	调用点开销极低	语义扩展与工具链耦合强	工程复现中以 native-like 固定字段二进制路径逼近其原生机制；Optbinlog 可扩展性更强，但热路径常数项更高
RT-Thread ulog	异步队列 + 多 backend	嵌入式设备在线调试	工程集成灵活	后台文本格式化与后端竞争	工程复现中以 native-like 异步队列路径逼近其原生机制；Optbinlog 在结构化与一致性方面更强

（续表 2.1）

系统	核心机制	典型使用场景	主要优势	主要代价	与 Optbinlog 的关系
OpenHarmony HiLog Lite	受限字段 + 轻量头	资源受限系统日志	资源占用较低	语义表达能力受限	工程复现中以 native-like 轻量头路径逼近其原生机制；Optbinlog 提供更完整字段语义和校验能力
Optbinlog	共享 schema + typed 编码 + CRC	多进程边缘网关/控制器	跨进程一致性、结构化可扩展、空间可控	初始化和通用编码固定成本	作为“语义完整 + 工程可迁移”的折中方案

如表2.1所示，Optbinlog 在设计上更偏向“语义完整、跨进程一致和工程可迁移”的折中路线，因此与追求极限热路径或系统级 tracing 的方案形成了明显分工。

1. 若目标是**内核态追踪**，Linux ftrace 仍是优先选择；若目标是**应用层结构化日志**，Optbinlog 更匹配。
2. 若目标是**极限热路径时间**，NanoLog/Zephyr logging 这类强调短热路径的方案更有优势；若目标是**语义可扩展与跨进程一致性**，Optbinlog 更稳定。
3. 若系统强调**运维生态兼容**，syslog 路径更易快速接入；若系统强调**嵌入式长期在线与空间效率**，Optbinlog 更具针对性。

2.4.2 二进制日志优化研究特征与 Optbinlog 对比

结合前文调研对象与公开实现路径，现有“二进制日志优化研究”大体可以归纳为三类技术取向：1. **热路径极短化**：通过编译期提取静态模板、运行期只记录动态参数，最大化降低调用点常数开销（典型如 NanoLog）。[8-10] 2. **并发写出解耦化**：通过无锁环形缓冲或 deferred 队列，将前台写入与后台落盘拆分，降低前台阻塞（典型如 ftrace/LTTng/Zephyr，以及 ETW 采集优化工具 Kellect）。

[1,3-4,15,21] 3. **后处理集中化**：把文本恢复、检索或压缩能力后移到离线/后台，以换取在线写入吞吐（典型如 NanoLog、集群日志无损压缩研究与 Binlog 工程方案）。[3,8,22-23]

对应地，这类研究的优化目标通常集中在两点：一是尽可能压缩单位日志写入代价，尤其是热路径延迟；二是在高并发条件下维持吞吐稳定，避免锁竞争或后台拥塞显著放大。相比之下，Optbinlog 并不追求单一指标上的极端最优，而是希望在较低写入开销的基础上，同时保留跨进程共享语义、记录级校验和在线恢复能力。

表 2.2 二进制日志优化研究特征与 Optbinlog 对比

研究/系统方向	优化重点与目标	与 Optbinlog 的关键差异
NanoLog 类编译期优化	目标是极限热路径时延；通过编译期拆分模板降低运行期开销	Optbinlog 不是纯离线恢复路径，而是保留共享 schema、在线结构化解码与记录级校验，热路径常数项更高但语义治理能力更强
ftrace/LTTng 类 tracing 缓冲	目标是高并发低侵入追踪；通过无锁/低锁缓冲提升采集吞吐	tracing 更偏系统观测而非业务日志语义一致性；Optbinlog 更强调应用层 event schema 与跨进程一致映射
Kellect（ETW 采集优化）	目标是“无丢失 + 低开销 + 实时”采集链路；优化采集与解析管线	更偏“日志采集器层”优化，不直接约束应用写入编码格式；Optbinlog 属于应用写入与共享格式层
Zephyr deferred/ulog 异步队列	目标是缩短前台路径、把代价后移到后台线程	Optbinlog 采用共享元数据 + 批量写出，不依赖独立后台日志线程，稳态行为更确定，但初始化与通用编码固定成本更明显

（续表 2.2）

研究/系统方向	优化重点与目标	与 Optbinlog 的关键差异
Lossless compression for large-scale cluster logs	目标是提升大规模日志压缩比与压缩效率，降低存储成本	属于“存储后处理层”优化，不直接降低单条写入热路径；Optbinlog 重点在写入阶段的结构化与一致性
Binlog（结构化二进制工程方案）	目标是元数据/数据分离与高效序列化，兼顾实时写入与后续解码	与 Optbinlog 在结构化方向接近，但 Binlog 更偏单进程库生态；Optbinlog 更强调共享格式文件与跨进程一致映射
Optbinlog（本文）	目标是“可复现、可校验、可扩展”的工程折中；强调共享 schema 与一致性	在 syslog/ulog 组可形成稳定综合优势；在 nanolog/zephyr 这类极短路径对照下仍需继续压缩单位记录常数项

如表2.2所示，现有工作的“突破点”主要分布在采集链路、后处理链路和结构化编码三类层面。NanoLog、ftrace、LTTng 与 Kellect 更强调采集路径本身，Lossless compression 更强调存储后处理，而 Binlog 更接近结构化编码库。[1,3-4,8,15,21-23] Optbinlog 的定位则更贴近嵌入式工程部署场景，即在语义对齐、并发一致、空间统计可追溯和记录级校验同时成立的前提下追求综合可用性。这一定位也解释了第 5 章中“对部分 peer 稳定占优、对极短路径 peer 仍有优化空间”的结果结构。

2.5 典型优化代码入口（可追溯）

为保证比较分析具有可追溯性，本文在代码级调研中选取了一组具有代表性的实现入口与官方格式说明，并据此建立“设计机制—实现路径—实验指标”之间的对应关系。这里所说的“代码入口”，并非简单的资料索引，而是用于识别各系统优化重点、解释实验差异来源以及限定复现边界的直接依据。

从分析作用看，这些入口主要服务于三个层面。其一，ring_buffer.c、log_msg.c、

RuntimeLogger.cc 等文件体现了写入热路径的组织方式，例如缓冲结构、前后台分工、模板展开和记录提交流程，因此可用于解释时间开销与吞吐表现的差异。其二，JOURNAL_FILE_FORMAT、ulog.c 以及 HiLog Lite 相关文档反映了记录头部、字段布局、存储约束和恢复方式，可用于分析单位记录字节开销及空间斜率差异。其三，这些入口共同界定了本文的复现边界：第 5 章并不是对所有外部系统进行完整移植，而是在可控环境下提取其代表性机制，构造 native-like 对照实现。[1,4-5,7-10,12-16]

基于上述处理，后文关于“共性不足”和“针对性设计”的论述都可以回溯到具体实现路径，而非停留在抽象层面的经验判断。这种写法有助于增强比较分析的证据基础，并为第 5 章实验结果的机制解释提供可核查支撑。

2.6 现有方案的共性不足

综合前述调研，可以看出不同系统虽然优化重点各异，但若从嵌入式多进程部署这一目标出发，仍存在四类共性不足：

1. **热路径优化目标较为单一**：部分方案重点压缩调用点延迟，但没有同时考虑结构化语义、共享元数据和记录级校验之间的平衡。
2. **共享元数据复用机制不足**：多进程环境下若各进程独立解析事件格式，启动与内存成本难以收敛，也不利于长期稳定运行。
3. **共享资源初始化缺乏显式一致性约束**：当格式文件或共享结构需要首次创建时，若缺少明确的状态机和协调过程，容易出现竞态。
4. **统一公平比较框架较少**：许多系统结论建立在各自原生语义和原生平台之上，缺少在相同语义体量与统一统计口径下的直接比较。

2.7 Optbinlog 的针对性设计

围绕上述缺口，Optbinlog 的设计并非简单叠加现有优化手段，而是针对嵌入式共享场景作出如下回应：

1. 用“事件格式文件 + 二进制编码”替代热路径中的字符串格式化，使稳态写入更多退化为字段遍历、编码与批量写出。
2. 用共享元数据文件持久化 tag 和 element 描述，并通过偏移访问实现跨进程复用，避免指针语义带来的映射不一致问题。

3. 用 INITIALIZING → INITIALIZED 状态机约束共享资源首次创建过程，保证单创建者与多复用者之间的并发语义清晰可检验。
4. 用统一实验脚本输出 mean/median/p95/std/95%CI 及 IQR 过滤后的统计结果，使理论推导、实现机制与实验结论能够在同一口径下对照。

2.8 复现边界与结论适用范围

为了避免结论外推过度，本文在此明确说明复现边界。整体工作遵循“代码阅读先行、实验复现跟进”的顺序开展：

1. 已完成 ftrace、Zephyr、NanoLog、ulog 等代表性系统的官方代码级阅读，并据此提取与本文比较直接相关的机制特征。
2. 受平台和环境限制，本文无法在当前工作站上完成所有外部系统的全栈编译与原生部署，尤其无法复现完整 Linux tracing 生态。
3. 已在可控环境中完成 nanolog_like、zephyr_deferred_like、ulog_async_like、hilog_lite_like 及其语义对齐版本的算法级复现，第 5 章所有定量结论均建立在该边界内。

上述约束在第 5 章实验方法中均有显式记录，用于区分“工程可复现边界”和“算法层结论边界”。因此，本文的排序结论仅在相应实现口径与实验条件下成立，不对未覆盖平台和未复现路径作超范围外推。

---

3 Optbinlog 系统设计与工程实现

本章围绕“从机制到实现、从实现到可验证结果”的逻辑展开。具体而言，先给出系统目标，再说明事件格式与共享元数据组织；随后描述写入与读取流程，并进一步讨论关键优化收敛路径、工程适配边界以及面向掉电场景的恢复机制。该安排对应第 4 章模型参数与第 5 章实验指标，可形成完整证据链。

3.1 设计目标

1. 降低单条日志热路径开销（时间）。
2. 降低长期写入空间消耗（空间）。

3. 支持多进程共享元数据与安全初始化（并发）。
4. 对齐嵌入式可部署需求（工程可落地）。

上述目标在后续章节中分别对应为可测指标：时间目标映射到 elapsed/end_to_end，空间目标映射到 total_bytes，并发目标映射到初始化竞争正确性与节点级稳定性，工程目标映射到跨场景复现与部署可迁移性。

3.2 事件格式与元数据结构

当前 Optbinlog 采用“共享元数据文件 + 进程内共享视图缓存 + 自描述二进制帧 + CRC32C(hw) + string-heavy 场景自动 varstr”的一体化实现路径；其事件定义思路与通用事件日志标签机制保持一致。[11,20]

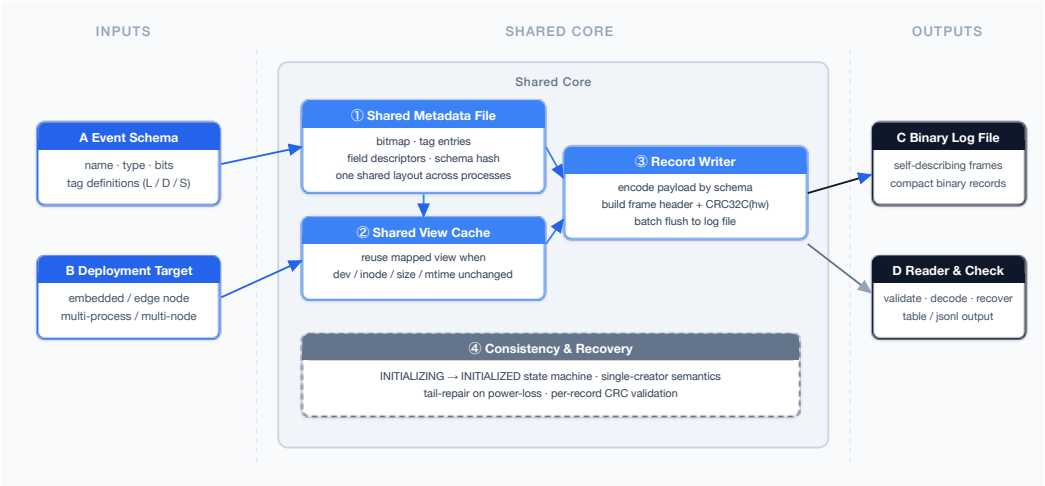

图 **3.1** Optbinlog 系统总体架构

如图3.1所示，系统以共享格式文件为中心，把“schema 解析、tag 组织、字段统计”前移到共享与缓存层，稳态写入只保留编码、帧构造、校验与批量落盘这四类关键操作。该组织方式直接对应第 4 章中的“固定项与边际项分离”思想。

事件定义采用 (name|type|bits) 形式，字段类型简化为 L/D/S：

1. L：定长整数，按 schema 指定字节数写入。
2. *D*：双精度浮点，固定写入 8 字节。
3. S：字符串字段；当前实现同时支持固定宽度字符串编码和 varstr 编码。

共享元数据文件内仍由以下结构组织：

1. OptbinlogSharedTag：共享文件头与初始化状态。

2. OptbinlogBitmap：tag 合法性位图。
3. OptbinlogEventTag：事件定义入口。
4. OptbinlogEventTagEle：元素类型与长度描述。

在共享文件之上，系统进一步引入了进程内共享视图缓存层。该缓存按共享文件的 dev/inode/size/mtime 判定是否可复用，并缓存：

1. 已映射的共享文件视图。
2. 每个 tag 的 OptbinlogTagCacheEntry。
3. schema 级统计量，例如 tag 数、总 payload 字节数、字符串字段总宽度、字符串字段数和最大字符串字段宽度。

这些统计量会直接参与写路径中的 varstr 自动决策，因此它们属于当前热路径优化的一部分。

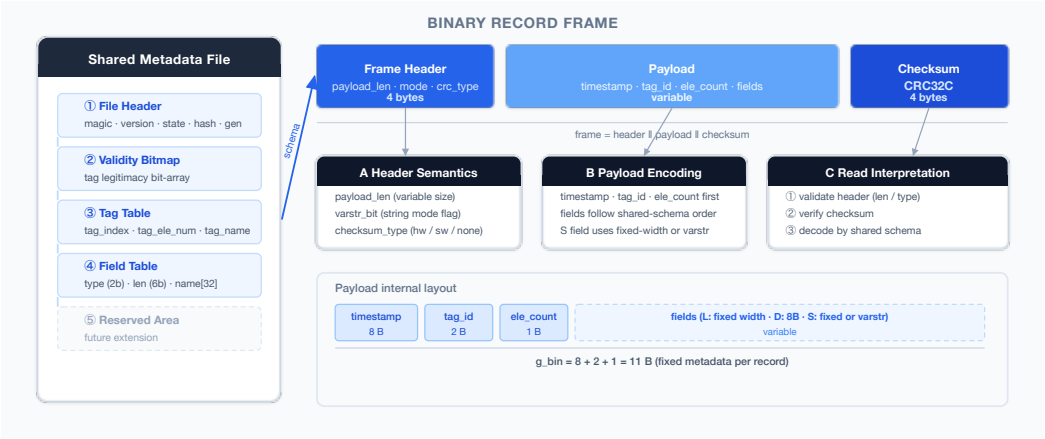

图 **3.2** Optbinlog 共享元数据与记录帧结构

如图3.2所示，左侧共享元数据布局保证“跨进程可移植”，右侧记录帧布局保证“按帧可校验、按 schema 可还原”，二者共同构成第 5 章实验统计可追溯的结构基础。

3.3 写入/读取路径

写入与读取遵循“元数据状态判定 → 编码/解码执行 → 一致性校验”的统一流程。

图 **3.3** Optbinlog 写入、读取与恢复流程

如图3.3所示，写入与读取并非两条割裂路径，而是由同一共享 schema 贯通。该流程设计使“语义对齐实验”和“记录级复核”能够使用同一套格式定义，减少统计口径漂移。

写入路径：1. 通过 acquire_shared_view() 获取共享文件视图；若共享文件未变化，则直接复用进程内缓存的 tag cache。2. 根据 schema 统计量判断是否启用 varstr；当字符串字段占比高或最大定长字符串字段较大时，默认切换到 varstr。3. 依据 tag cache 顺序编码 payload。4. 构造 4 字节自描述帧头，其中同时编码 payload_len、varstr 标志位和校验类型。5. 对单条记录计算校验值；默认采用 CRC32C(hw)，仅在硬件不可用时回退到软件实现。6. 将 frame_header + payload + checksum 写入用户态缓冲区，按批次落盘。

读取路径：1. 读取 4 字节帧头，并解析 payload_len、varstr 标志位与 checksum_type。2. 读取 payload 与校验字段，并按帧头声明的校验类型执行校验。3. 按共享格式解码：整数和浮点按固定宽度读取，字符串则根据 varstr 标志选择“长度前缀 + 数据”或“固定宽度截断/补零”方式。4. 将解码后的记录交给统一回调接口。

因此，当前记录格式可写作：

$$\text{frame} = \text{frame_header} \parallel \text{payload} \parallel \text{checksum}$$

其中：

$$\text{frame_header} = \text{payload_len} \cup \text{varstr_bit} \cup \text{checksum_type}$$

3.4 关键优化路径与收敛

为突出与最终结果直接相关的设计取舍，本文将实现演化归纳为三个阶段：

1. **结构一致性阶段**：将“每事件独立结构”统一为“共享结构 + 位图 + 偏移表”，并将跨进程共享从指针语义改为偏移语义，解决映射基址不一致导致的可移植性问题。
2. **热路径压缩阶段**：将 tag 查找改为常数级索引路径，引入进程内共享视图缓存与 tag cache，把共享文件 reopen、tag 遍历与 schema 统计迁出稳态写路径。
3. **可靠性与空间协同阶段**：从软件 CRC32 迭代为默认 CRC32C(hw)，引入自描述帧头与 varstr 自动策略，并通过初始化状态机与 init-race 验证保证“单创建者 + 多复用者”并发语义。

对应到当前版本，系统收敛点可概括为：

1. **可靠性不降、校验成本下降**：保留每条记录校验语义，同时以 CRC32C(hw) 降低热路径校验开销。
2. **共享元数据收益进入稳态**：共享视图缓存使 schema 解析与 tag 遍历不再重复进入每次写入。
3. **字符串冗余持续收缩**：string-heavy 场景下自动启用 varstr，减少固定宽度补零与无效拷贝。

3.5 工程场景适配性

1. 小样本 N：共享文件打开、共享视图校验、文件打开和缓冲准备等固定项占比仍然较高，时间优势不一定立刻出现。
2. 中大样本 N：固定项被摊薄，自描述帧头和 CRC32C(hw) 的额外代价收敛到常数级，binary 的时间和空间优势更稳定。
3. string-heavy schema：自动 varstr 会显著减少补零写入和冗余字节，因此空间优势更容易显现。
4. 高并发 D：即使校验和 schema 解析成本被压低，调度、文件竞争和写出路径仍会主导吞吐峰值与回落点。
5. 多进程/多节点部署：共享 schema、进程内缓存复用和节点级初始化竞争正确性已经形成闭环，因此该设计更贴近真实工程部署。

因此，第 4 章将把上述实现行为进一步抽象为 $A_m/C_m/B_m/W_m$ 等参数形式，便于与第 5 章实验结果逐项对照。

3.6 读取与复核能力设计

为提升实验结果的可复核性，本文在写入与解码库之外提供了 optbin-log_read 读取工具，用于对 “共享格式文件 + 二进制日志文件” 进行直接验证。该工具具备三项能力：1. 按 tag 与字段名输出记录，避免只看到匿名值序列。2. 支持 table/jsonl 两种格式，便于命令行排错与自动化分析脚本复用。3. 支持 --limit 与 --summary，可快速截取样本并统计各 tag 分布。

该工具将 “统计结果” 与 “底层记录” 建立起可追溯映射，使第 5 章中的对比结论可回退到记录级证据，而非仅停留在汇总表与图形层面。

3.7 断电场景下的截断恢复机制

嵌入式设备掉电时，最常见的损坏形态是 “尾帧半写入”：帧头、payload 或校验字段未完整落盘。若在该状态下直接续写，新数据将附着在损坏尾部之后，读取阶段会触发 truncated 或 crc mismatch，进而影响后续完整日志的可读性。

针对该问题，本文在写入路径中采用 “尾部恢复后再续写” 的处理策略，其核心原则如下：

1. 只保留**最后一条完整帧之前**的数据；
2. 对损坏尾部执行一次性截断；
3. 截断后再恢复 append 写入。

实现流程可概括为：

1. 顺序扫描 run.bin，按 `frame_header → payload → checksum` 逐帧校验；
2. 记录 “最后一条完整帧结束偏移” valid_end；
3. 一旦遇到以下任一情况即判定尾部损坏：frame length 非法、checksum_type 非法、payload 截断、CRC 截断、CRC 不一致；
4. 将文件截断到 valid_end；
5. 在截断完成后恢复 append，确保后续日志可连续写入。

工程上，append 模式在进入稳定写入前会先执行尾部一致性检查；当检测到半帧时先裁剪坏尾，再进入续写阶段。这样在“供电恢复/进程重启”后，系统能够优先消除上一次掉电遗留的尾部损坏，避免错误链式扩散。

该机制的语义边界为“可检测、可修复、可继续”：

1. 正在写入且被断电打断的那条日志视为不可恢复数据，将被丢弃；
2. 已完整落盘的历史帧全部保留；
3. 修复后后续日志可继续追加并被正常解码。

因此，Optbinlog 在可靠性层面从“仅检测损坏”扩展为“检测 + 恢复 + 续写”的闭环处理，更符合真实嵌入式供电波动场景。

---

4 理论模型与数学推导

本章先做抽象推导，不代入具体事件文件数值。为保证推导可解释且可验证，本文采用以下前提：在单次统计窗口内事件分布近似稳定；各模式在同一场景下使用一致的输入语义；模型参数以期望意义解释而非单点极值解释。

4.1 符号定义

1. N：总日志条数。
2. D：并发设备数（或并发进程数）。
3. r：每设备日志条数，满足 $N = D \cdot r$。
4. k：事件类型（tag）索引，$k \in \{1, \dots, T\}$。
5. p_k：事件类型 k 的概率，满足 $\sum_k p_k = 1$。
6. e_k：事件 k 的字段数。
7. $l_{k,j}$：事件 k 第 j 个字段编码字节数。
8. $L_k = \sum_{j=1}^{e_k} l_{k,j}$：事件 k 的有效载荷字节数。
9. A_m：模式 m 的固定时间开销（初始化、打开文件、预热）。
10. C_m：模式 m 的单条边际时间开销。
11. B_m：模式 m 的单条边际空间开销。
12. $S_{0,m}$：模式 m 的固定空间开销（例如共享元数据）。
13. Q_m：吞吐（records/s）。

除特别说明外，本章的“空间”均对应实验统计中的 total_bytes：对 binary 为“日志文件 + 共享格式文件且共享文件只计一次”，对其他模式为其实际输出文件大小；“时间”默认对应单轮写入阶段的均值趋势，若切换到 end_to_end 口径，则把进程/节点启动与调度准备吸收到固定项中。

4.2 从算法与数据结构到单条代价

为避免“只看结果不看机制”，先把每种模式拆成操作级成本。以下表达式均以单条记录期望开销为核心，再扩展到总量关系。

4.2.1 text

单条记录包含：1. 读取输入字段。2. snprintf/拼接格式串。3. 写入用户态缓冲并最终落盘。

时间可写为：

$$t_{\text{text}}(k) = t_{\text{parse}}(k) + t_{\text{fmt}}(k) + t_{\text{write}}(\text{bytes}_{\text{text}}(k))$$

空间可写为：

$$b_{\text{text}}(k) = b_{\text{prefix}} + b_{\text{delim}} + b_{\text{num}}(k) + b_{\text{str}}(k)$$

其中 $b_{\text{num}}(k)$ 是数字转十进制后的字符长度，随数值大小缓慢变化。

4.2.2 binary（optbinlog）

当前 optbinlog_binlog_write() 的稳态路径可分为四步：1. 复用共享视图与 tag cache，并按 tag_id 完成常数级查找。2. 按字段类型编码 payload。3. 根据校验模式计算记录校验。4. 将记录写入用户态批量缓冲区，缓冲区满后统一写出。

与代码一致，单条记录的物理布局为：

$$\text{frame} = \text{frame_header}_{4B} \parallel \text{payload} \parallel \text{checksum}_{4B}$$

其中 payload 并不只是 schema 字段本身，还额外包含：

$$\text{payload} = \text{timestamp}_{8B} \parallel \text{tag_id}_{2B} \parallel \text{ele_count}_{1B} \parallel \text{fields}$$

记记录内部固定元数据开销为：

$$g_{\text{bin}} = 8 + 2 + 1 = 11$$

若字符串字段按固定宽度写出，则 payload 中 schema 字段的有效载荷仍记为 L_k；若 schema 触发 varstr，则对应地记为 $\tilde{L}_k$。于是单条时间可写为：

$$t_{\text{bin}}(k) = t_{\text{lookup}} + t_{\text{encode}}(k) + t_{\text{checksum}}(k) + \bar{t}_{\text{write}}(b_{\text{bin}}(k))$$

其中 $\bar{t}_{\text{write}}(\cdot)$ 表示“经用户态缓冲摊薄后的平均写出代价”，比逐条系统调用更接近当前实现。

单条空间则可写为：

$$b_{\text{bin}}(k) = 4 + g_{\text{bin}} + \tilde{L}_k + 4$$

固定空间只在共享格式文件层面体现，因此有：

$$S_{0,\text{bin}} = S_{\text{shared}}$$

若 schema 触发 varstr，则字符串字段的有效载荷不再按固定宽度写出，而是写为“长度前缀 + 实际长度”：

$$\tilde{L}_k = \sum_{j \in \mathcal{N}_k} l_{k,j} + \sum_{j \in \mathcal{S}_k} (2 + \hat{l}_{k,j}), \qquad \hat{l}_{k,j} \le l_{k,j}$$

还需说明两点与代码实现直接对应的口径：1. 默认 CRC32C(hw) 覆盖的是“帧头 + payload”；兼容模式下 CRC32 覆盖 payload，none 则不做校验。2. 共享文件的打开、映射与 tag cache 构建只在视图失效时重新执行，因此它们主要进入固定项 A_{bin}，而不是每条记录的边际项。

4.2.3 syslog / ftrace

抽象为“系统日志接口 + 内核路径”：

$$t_{\text{sys}}(k) = t_{\text{format_sys}}(k) + t_{\text{syscall}} + t_{\text{kernel_path}} + t_{\text{write_kernel}}$$

$$t_{\text{ftrace}}(k) = t_{\text{format_ftrace}}(k) + t_{\text{trace_write}} + t_{\text{ring_or_file_path}}$$

空间：

$$b_{\text{sys}}(k) = b_{\text{sys_header}} + b_{\text{sys_payload}}(k)$$

$$b_{\text{ftrace}}(k) = b_{\text{ftrace_header}} + b_{\text{ftrace_payload}}(k)$$

4.2.4 nanolog_like / zephyr_deferred_like / ulog_async_like / hilog_lite_like

nanolog_like（固定字段二进制）：

$$t_{\text{nano}} = t_{\text{pack_fixed}} + t_{\text{memcpy}} + t_{\text{write}}$$

$$b_{\text{nano}} = \text{const}$$

zephyr_deferred_like（前台入队 + 后台批刷）：

$$t_{z,\text{front}} = t_{\text{enqueue}} + t_{\text{signal}}$$

$$t_{z,\text{back}} = \frac{t_{\text{dequeue}} + t_{\text{batch_write}}(k_b)}{k_b}$$

$$t_z = t_{z,\text{front}} + t_{z,\text{back}}$$

$$b_z = b_{\text{queue_node}} + b_{\text{fixed}} + \frac{b_{\text{batch_meta}}}{k_b}$$

ulog_async_like（异步队列 + 文本 backend）：

$$t_{\text{ulog}} = t_{\text{enqueue}} + t_{\text{cond_signal}} + \frac{t_{\text{dequeue}} + t_{\text{format_text}} + t_{\text{backend_write}}}{k_b}$$

$$b_{\text{ulog}} = b_{\text{queue_node}} + b_{\text{text_payload}}$$

hilog_lite_like（受限字段 + 轻量头）:

$$t_{\text{hilog}} = t_{\text{compact_format}} + t_{\text{write}}(h_{\text{lite}} + p_{\text{lite}})$$

$$b_{\text{hilog}} = b_{\text{header_lite}} + b_{\text{payload_lite}}$$

4.3 空间随 N 的详细推导

4.3.1 期望单条字节

对任意模式 m，定义:

$$B_m = \mathbb{E}[b_m(k)] = \sum_k p_k\, b_m(k)$$

则总空间:

$$S_m(N) = S_{0,m} + NB_m \tag{4.1}$$

这是严格的一阶线性模型，斜率为 B_m，截距为 $S_{0,m}$。见公式（4.1）所示，后续空间分析都可以归结为“固定项 + 单条斜率”两部分。

4.3.2 binary 的显式表达

由 4.2.2 得:

$$B_{\text{bin}} = 8 + g_{\text{bin}} + \sum_k p_k \tilde{L}_k$$

$$S_{\text{bin}}(N) = S_{\text{shared}} + N\left(8 + g_{\text{bin}} + \sum_k p_k \tilde{L}_k\right) \tag{4.2}$$

其中，$g_{\text{bin}} = 11$ 对应 payload 内部固定元数据，外层的两个 4B 分别对应帧头和校验字段。见公式（4.2）所示，binary 的总空间同时受“每条记录固定项”和“共享格式文件固定项”影响。

4.3.3 节省率与导数

binary 相对模式 x 的空间节省率:

$$\eta_{\text{bin}/x}(N) = 1 - \frac{S_{\text{bin}}(N)}{S_x(N)}$$

若 $S_{0,x} = 0$，则

$$\eta_{\text{bin}/x}(N) = 1 - \frac{B_{\text{bin}} + S_{\text{shared}}/N}{B_x}$$

求导：

$$\frac{d\eta_{\text{bin}/x}}{dN} = \frac{S_{\text{shared}}}{N^2 B_x} > 0$$

说明 N 越大，S_{shared}/N 越小，binary 空间优势越稳定。这说明只要边际斜率满足优势条件，共享文件这一固定项就会随着 N 增大而被持续摊薄。

4.3.4 空间交叉点

当比较 bin 与 x：

$$S_{\text{bin}}(N) \leq S_x(N)$$

$$S_{\text{shared}} + NB_{\text{bin}} \leq S_{0,x} + NB_x$$

若 $B_x > B_{\text{bin}}$，可得交叉点：

$$N^*_{\text{space}}(\text{bin}, x) = \frac{S_{\text{shared}} - S_{0,x}}{B_x - B_{\text{bin}}} \tag{4.3}$$

当 $N > N^*_{\text{space}}$ 时，binary 空间一定优于 x。因此，空间交叉点的判断可以直接按公式（4.3）所示进行。

4.4 时间随 N 的详细推导

4.4.1 从单条时间到总时间

定义：

$$C_m = \mathbb{E}[t_m(k)] = \sum_k p_k \, t_m(k)$$

则总时间：

$$T_m(N) = A_m + NC_m \tag{4.4}$$

该形式直接对应实验中的 elapsed/write_only/end_to_end 指标。见公式（4.4）所示，总时间同样由固定项和边际项组成。若统计口径切换到 end_to_end，则进程/节点启动、文件准备与调度等待也并入 A_m。

4.4.2 吞吐公式与单调性

$$Q_m(N) = \frac{1000N}{T_m(N)} = \frac{1000}{C_m + A_m/N} \tag{4.5}$$

对 N 求导：

$$\frac{dQ_m}{dN} = \frac{1000A_m}{N^2\left(C_m + A_m/N\right)^2} > 0$$

说明在模型中吞吐随 N 单调上升并渐近：

$$\lim_{N\to\infty} Q_m(N) = \frac{1000}{C_m}$$

在 $A_m > 0$ 与 $C_m > 0$ 条件下，吞吐增速会随 N 增大而下降，表现为“先快后慢”的收益递减，这与实验中大 N 后增益变缓一致。因此，公式（4.5）给出了第 5 章解释“吞吐为何先快后慢趋于平台”的统一表达。

4.4.3 时间交叉点（binary 何时优于其他模式）

比较 bin 与 x：

$$T_{\text{bin}}(N) \le T_x(N)$$

$$A_{\text{bin}} + NC_{\text{bin}} \le A_x + NC_x$$

当 $C_x > C_{\text{bin}}$ 时：

$$N^*_{\text{time}}(\text{bin}, x) = \frac{A_{\text{bin}} - A_x}{C_x - C_{\text{bin}}} \tag{4.6}$$

解释：1. 若 A_{bin} 较大（共享初始化重），小 N 可能不占优。2. 随 N 增大，NC 项主导，只要 $C_{\text{bin}} < C_x$，最终会跨过交叉点。

因此，某一模式何时在时间上完成反超，可按公式（4.6）所示理解。

4.4.4 C_m 的字段复杂度展开

为把“常数项差异”写清楚，可把边际开销展开到字段数级别：

$$C_m = c_{m,0} + c_{m,1}\mathbb{E}[e_k] + c_{m,2}\mathbb{E}[L_k] + c_{m,3}\mathbb{E}[\text{syscall}_m]$$

其中：1. $\mathbb{E}[e_k]$ 决定字段遍历成本（编码循环次数）。2. $\mathbb{E}[L_k]$ 决定 memcpy、校验和写出字节相关成本。3. $\mathbb{E}[\text{syscall}_m]$ 反映模式是否显著依赖系统日志接口或内核路径。

结合当前复现代码，可得到更贴近实现的解释：1. text_semantic_like 主要承担字段格式化与文本展开成本。2. binary 的边际项主要由 typed encode、校验和批量写出摊销组成，固定项则额外包含共享视图与 tag cache 准备。3. nanolog_like / zephyr_deferred_like 走固定宽度打包路径，边际项通常较低，但 zephyr_deferred_like 还会受到批刷组织方式影响。4. ulog_async_like / syslog / ftrace 会显著暴露文本展开、系统接口或内核路径的常数开销。5. hilog_lite_like 通过轻量头和受限消息体压缩了单条记录规模，因此常落在二进制直写组与文本系统组之间。

4.5 多设备 D 的详细推导

4.5.1 总时间分解

设每设备写 r 条：

$$E_m(D,r) = A_{\text{spawn}}(D) + A_m + rC_m + W_m(D,r) \tag{4.7}$$

见公式（4.7）所示，多设备总时间除了单机边际项外，还需要显式计入并发等待项。若观察的是 write_only 指标，则 $A_{\text{spawn}}(D)$ 会明显缩小；若观察 end_to_end，它则不能忽略。

其中 W_m 为竞争等待项（锁、队列、内核资源、I/O 饱和）。

4.5.2 队列化近似（解释吞吐饱和）

把共享瓶颈抽象成单服务台，服务率 μ_m（records/s），到达率 $\lambda = D\nu$（ν 为单设备提交率）。

稳定条件：

$$\rho = \frac{\lambda}{\mu_m} < 1$$

等待时间近似随 ρ 增大而快速上升，可抽象为：

$$W_m(D, r) \sim r f_m(\rho), \qquad f_m'(\rho) > 0, \qquad f_m''(\rho) > 0$$

因此当 D 接近

$$D_{\text{sat},m} \approx \frac{\mu_m}{\nu}$$

吞吐进入平台区；超过后出现明显回落。这里的“服务台”不一定是真实队列，也可以对应共享文件写出通道、后台批刷线程或内核 tracing 接口；该抽象的意义在于用同一符号解释不同模式的饱和与回落。

4.5.3 常用二次近似

在工程拟合中常写作：

$$E_m(D, r) \approx \beta_m + rC_m + \alpha_m D + \gamma_m D^2$$

总吞吐：

$$Q_m(D, r) = \frac{1000Dr}{E_m(D, r)}$$

当 $\gamma_m > 0$ 时，D 过大必然导致分母增长过快，出现峰值后下降。

进一步对 D 求导：

$$\frac{dQ_m}{dD} \propto \left(\beta_m + rC_m + \alpha_m D + \gamma_m D^2\right) - D\left(\alpha_m + 2\gamma_m D\right)$$

整理得：

$$\frac{dQ_m}{dD} \propto \beta_m + rC_m - \gamma_m D^2$$

故吞吐峰值点近似为：

$$D_{\text{peak},m} = \sqrt{\frac{\beta_m + rC_m}{\gamma_m}} \tag{4.8}$$

这给出了可直接与实验扫描结果比对的“理论峰值设备数”。其中，峰值设备数可直接按公式（4.8）与实验扫描结果对照。

4.5.4 空间随 D

$$S_m(D, r) = S_{0,m} + DrB_m$$

因此空间对 D 严格线性，差异主要由斜率 B_m 与截距 $S_{0,m}$ 决定。这一点与当前实验统计口径一致：在记录规模 r 固定时，设备数增加只会线性放大记录总量，而不会改变每条记录的平均编码体积。

4.6 模式间理论排序与可检验预测

4.6.1 异语义比较（最短路径口径）

在工程近似层中，各模式的输入语义和存储组织并不完全一致，因此本节只讨论“机制上更可能出现的方向”，而不把它写成固定全序。

从当前复现代码出发，可得到三条较稳定的预期：1. 固定宽度二进制直写组（如 nanolog_like、zephyr_deferred_like）通常拥有更低的单位记录空间与更短的边际写入路径。2. 文本展开或系统路径更重的模式（如 text/syslog/ftrace/ulog_async_like）通常拥有更高的常数项。3. binary 与 hilog_lite_like 处于二者之间：前者保留共享 schema、自描述帧和校验语义，后者保留轻量头和受限消息体，因此二者的排序会更依赖具体 schema 与统计口径。

可检验预测：1. nanolog_like 在时间与吞吐上常呈数量级优势。2. hilog_lite_like 在“受限语义”下应明显优于 text/syslog。3. ulog_async_like 在吞吐未饱和且后台轻负载时有机会受益；若后台文本后端成为瓶颈，可能慢于 binary/text。4. 上述优势不代表语义等价下仍保持同倍率。

4.6.2 语义对齐比较（公平口径）

本文所说的“语义对齐”，并不是把所有 peer 强行改写成同一种记录格式，而是在**同一组字段语义、同一组字段值与同一平台**下，让 Optbinlog 与 peer 分别走各自的 native-like 复现路径。于是：

1. binary 的差异主要来自共享 schema、frame 组织、per-record CRC 以及字段遍历。
2. peer 的差异主要来自其 native-like 记录布局、批刷/后台路径、文本展开与系统接口路径。

因此，公平口径下的空间关系不再由“语义体量不同”主导，而主要由各模式的**单位记录常数项**主导。对于 nanolog_like / zephyr_deferred_like 这类紧凑二进制 peer，B_{peer} 往往长期小于 B_{bin}；对于 ulog_async_like / hilog_lite_like / syslog 这类仍带有消息头、文本展开或系统路径附加开销的 peer，B_{bin} 更可能在较小 N 后越过交叉点。

C_{peer} 与 C_{bin} 的差异同样主要来自实现常数，包括编码循环、缓存命中、批刷组织、格式化路径与校验实现，而不再主要来自“消息语义更多或更少”的偏差。

可检验预测：1. 时间差异将由各模式的原生热路径决定，而不是由语义字段数决定。2. 空间差异会在同一语义组内呈现稳定斜率，并在 N 增长时显现是否存在交叉点。

4.7 与第 5 章指标的映射关系

为了让推导可被实验验证，建立公式到指标的映射：

1. $T_m(N)$ 对应 elapsed_ms 或 end_to_end_ms 的均值趋势。
2. Q_m 对应 throughput_rps / throughput_e2e_rps。
3. S_m 对应 total_bytes。
4. N^*_{time} 与 N^*_{space} 对应“某场景下 binary 由劣转优”的阈值。
5. $D_{\text{sat},m}$ 对应“多设备扫描中吞吐峰值后回落”的拐点。

本章给出的是“由算法与数据结构直接推出的理论趋势”；第 5 章再代入实测数据检验这些趋势是否成立，以及偏离原因。

4.8 本章小结

本章完成了三项工作：一是把不同日志模式统一到可比较的时间与空间参数形式；二是给出 N 与 D 两个核心自变量下的交叉点、峰值点与线性斜率含义；三是建立理论参数与实验指标的映射关系。基于这些结果，第 5 章将按“单机 → 本地多设备 → 真实多设备模拟 → 日志量扫描”的顺序进行逐层验证。

---

5 实验设计、结果与理论一致性分析

5.1 实验设计与评估口径

本章按照“实验口径—结果呈现—理论对照—机制解释”的顺序展开。主证据采用最新一版严格语义对齐数据，统一比较对象为 text_semantic_like / binary / peer native mode，并覆盖以下三类场景。[20] 1. 单机高负载（single）。2. 本地多设备模拟（$D = 5, 10, 20, 50$ 扫描）。3. 节点级真实多设备代理场景，采用 5/10/15/20 节点扫描，每节点一台设备。

此外，本文同步报告两组初始化竞争结果，用于验证共享格式文件在本地多进程和节点级并发条件下的正确性。统计口径统一为 mean/median/p95/std/95%CI，并先按 IQR 1.5 倍规则过滤异常值，再输出时间、空间、吞吐与改进百分比，以保证不同场景之间具有可比性。

本轮核心原则为：**Optbinlog 对齐 peer 语义，peer 保持原生存储路径；所有模式共享同一字段值生成器；共享格式文件在集群空间统计中仅计算一次。**在此基础上，本文增加了 $N = 1 \sim 100000$ 的空间扫描，用于回答“何时发生空间反超”。消融数据仅用于解释机制，不作为主结论依据。

为保证分析口径可追溯，本章额外固定“对比对象、运行平台与数据量”三类约束。尤其在节点级真实多设备代理场景中，本文采用“**全节点共享同一 shared_eventtag.bin 格式缓存文件，节点结果文件独立输出**”的方式，重点观察共享元数据在并发访问下的竞争与收益释放。

表 5.1 本章统一实验设计与评估口径

维度	本文实验口径
对比对象	text_semantic_like（文本基线）、binary（Optbinlog 最终路径）；single/multi peer：syslog、nanolog、zephyr、ulog、hilog；l1 peer：syslog、ftrace、nanolog、zephyr、ulog、hilog；peer 侧保持原生存储路径
单机/本地多设备平台	macOS 26.3（Darwin 25.3.0，ARM64）
真实多设备平台	Lima Ubuntu 24.04 ARM64 虚拟机集群；最多 20 节点；节点 CPU 为 2 vCPU（aarch64）
校验实现与硬件路径	binary 默认每条记录 CRC32C(hw)；L1 节点 aarch64 CPU flags 包含 crc32，基准按 -march=armv8-a+crc 编译
真实多设备数据量	每节点 records=20000；节点规模 5/10/15/20；repeats=3，warmup=1
真实多设备总样本	六个语义组 × 三种模式 × 节点扫描后，有效统计样本约 54,000,000 条日志（不含 warmup）
真实多设备总日志字节	有效轮次聚合约 4.601 GiB（按 cluster_total_bytes 汇总）
真实多设备空间口径	集群总空间；共享格式文档在每个模式下仅计算一次
空间交叉点附加扫描	records 从 1 到 100000 共 13 个点；repeats=2；指标为 total_bytes

后续 5.2 ~ 5.7 的表格与图像分析均以表5.1为统一口径，确保各场景对比结论可直接互相参照。

图像呈现遵循“主图承载结论、辅图解释机制”的统一规则，即每组实验至少保留一张能够直接支撑判断的主图，并辅以差值图或分解图说明原因，避免只

有曲线堆叠而缺少结论指向。

本文以严格公平层作为主结论来源，其中 syslog 覆盖 single/multi/l1 三类场景，ftrace 覆盖真实多设备场景。工程近似层仅用于补充系统全链路代价解释。该分工可将“实现差异”与“平台路径差异”有效分离。

5.2 初始化竞争正确性（前置验证）

在进入性能对比之前，本文先验证共享格式文件并发初始化的正确性，以确保后续 single/multi/l1 指标建立在一致的并发语义之上。[1,4,15]

1. 本地多进程初始化竞争。

   平均耗时为 12.337 ms，$p95$ 为 13.679 ms，平均创建成功进程数为 1.0，平均复用已存在共享文件成功进程数为 9.0。

2. 节点级真实多设备代理场景下的初始化竞争。

   全局平均耗时为 6480.706 ms，$p95$ 为 6495.396 ms，平均单创建者数量为 1.0，平均失败节点数为 0.0。

两组结果均稳定满足“单创建者、其余进程或节点复用已初始化共享文件”的正确性约束，说明共享元数据机制在本地与节点级并发下均保持一致行为。基于该前置验证，后续性能实验按“单机基线 → 本地多设备 → 真实多设备 → 日志量扩展”顺序展开，以避免把初始化异常误判为性能差异。

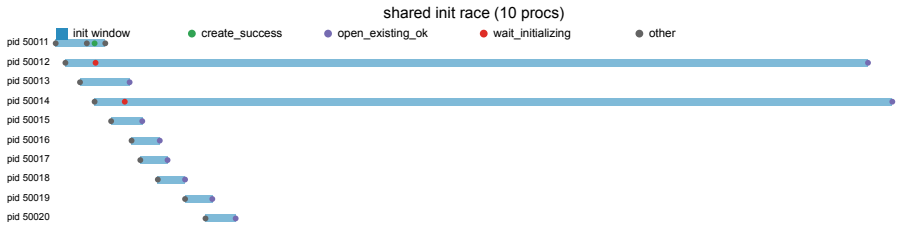

图 **5.1** 本地多进程初始化竞争时序图

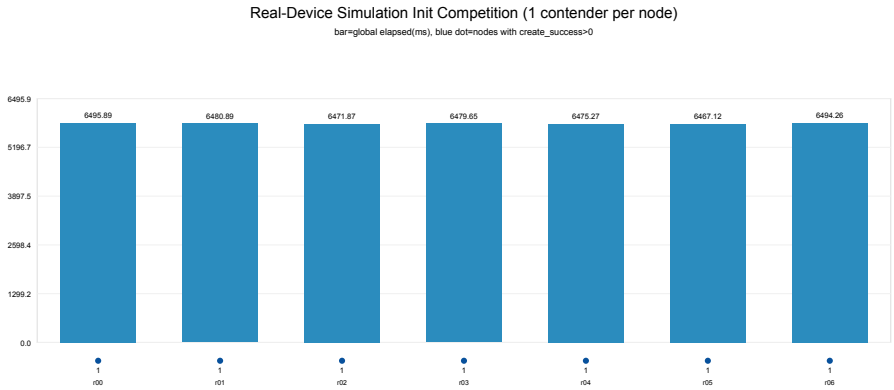

图 **5.2** 真实多设备节点级初始化竞争总览图

5.3 单机场景：严格对齐下的基线关系

本节结果来自严格语义对齐套件的合并汇总文件。三类模式统一采用同一 eventlogst_semantic_* 语义组、同一生成器产生的字段值，以及同一平台；其中 binary 采用 “共享 schema/tag cache + per-record CRC32C(hw) + string-heavy auto-varstr” 的当前实现，而 peer 侧保持各自原生存储路径。

表 **5.2** 单机高负载下 binary 相对 text 的结果

分组	binary 相对 text 时间%	binary 相对 text 空间%	binary 相对 text 吞吐%	结论
syslog	+73.29%	+45.75%	+274.20%	相较 text 呈稳定显著正收益
nanolog	+93.01%	+69.79%	+1335.62%	相较 text 呈稳定显著正收益
zephyr	+93.50%	+69.52%	+1438.78%	相较 text 呈稳定显著正收益
ulog	+93.72%	+68.96%	+1487.27%	相较 text 呈稳定显著正收益
hilog	+93.78%	+71.95%	+1508.37%	相较 text 呈稳定显著正收益

表 5.3 单机高负载下 binary 相对 peer 的结果

分组	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	结论
syslog	+74.17%	+45.75%	+287.00%	三指标同向为正，优势稳定
nanolog	+21.21%	-41.76%	+27.37%	时间与吞吐转正，空间仍为负区
zephyr	+25.12%	-41.76%	+33.34%	时间与吞吐转正，空间劣势持续
ulog	+83.73%	+59.43%	+515.11%	三指标同向为正，综合优势显著
hilog	+77.29%	+32.61%	+340.10%	三指标同向为正，综合优势明确

如表5.2和表5.3所示，单机场景呈现稳定的双层分化特征。其一，binary vs text_semantic_like 在五个语义组上均为正收益，说明文本格式化与字符串拼接在严格对齐条件下仍是主要开销来源。其二，在 peer 原生路径对照下，binary 对 nanolog/zephyr 的时间与吞吐已转入正区，但空间仍处于负区；对 syslog/ulog/hilog 则三项指标总体为正。该结果表明，在单机稳态阶段 Optbinlog 已具备时间优势，但 frame/CRC 常数项仍限制其对紧凑二进制 peer 的空间反超能力。

因此，本章后续结论以严格公平层为主，并以同一 schema、同一字段值、同一平台下的结果解释 Optbinlog 相对 peer 的本征开销差异。

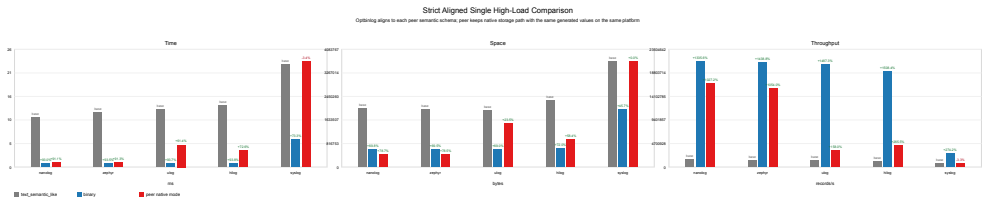

图 **5.3** 严格语义对齐单机高负载总览

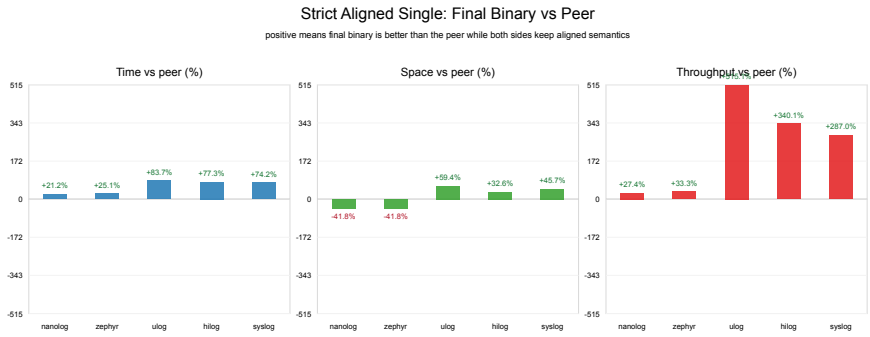

图 **5.4** 严格语义对齐单机场景中 binary 相对 peer 的差值

5.4 本地多设备：竞争项放大后的趋势

本节结果同样来自 final_aligned_summary.json，但切换到 multi 部分，并对 $D = 5, 10, 20, 50$ 进行扫描。该场景保持平台不变，只让设备数增加，因此能够单独观察 $W_m(D, r)$ 对趋势的影响。

表 **5.4** 本地多设备模拟下 binary 相对 peer 的扫描结果

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
syslog	5	-43.04%	-11.31%	+45.68%	低并发阶段时间与吞吐为负
syslog	10	+9.31%	+10.28%	+45.72%	中并发转入稳定正收益

（续表 5.4）

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
syslog	20	+7.35%	+7.92%	+45.73%	中高并发保持正区
syslog	50	+9.79%	+10.79%	+45.74%	高并发仍处正区
nanolog	5	+2.94%	+1.88%	-42.12%	时间吞吐小幅为正，空间负区
nanolog	10	-1.48%	-1.41%	-41.90%	中并发回落至零点附近
nanolog	20	+0.75%	+0.76%	-41.78%	小幅回正但空间未变
nanolog	50	+14.47%	+11.33%	-41.71%	时间吞吐转强，空间仍负
zephyr	5	-5.51%	-4.93%	-42.14%	低并发轻度负收益
zephyr	10	+1.00%	+1.01%	-41.90%	中并发接近持平并微正
zephyr	20	-7.87%	-5.89%	-41.78%	竞争放大后再次转负

（续表 5.4）

分组	设备数 D	binary 相对 peer 时间%	binary 相对 peer 吞吐%	binary 相对 peer 空间%	现象
zephyr	50	+13.53%	+10.95%	-41.71%	高并发回升为正
ulog	5	+1.19%	+1.17%	+59.32%	低并发即小幅转正
ulog	10	+11.74%	+12.15%	+59.39%	中并发优势扩大
ulog	20	+6.74%	+7.05%	+59.42%	中高并发保持正区
ulog	50	-10.21%	-6.32%	+59.44%	高并发时间吞吐回落
hilog	5	+1.92%	+2.03%	+32.41%	低并发小幅领先
hilog	10	-1.88%	-1.83%	+32.53%	中并发短暂转负
hilog	20	+0.67%	+0.66%	+32.59%	中高并发近持平
hilog	50	+1.67%	+1.49%	+32.63%	高并发小幅回正

如表5.4所示，本地多设备阶段最显著的现象是：时间与吞吐围绕零点波动，而空间差异基本保持稳定。nanolog/zephyr 的空间差值长期约为 -42%，syslog/ulog/hilog 分别稳定在约 +46%、+59% 与 +33%。与旧轮次相比，syslog 在

$D = 5$ 出现明显负区（时间 -43.04%、吞吐 -11.31%），但在 `D>=10` 后重新转正；ulog 在 $D = 50$ 也出现阶段性回落。这表明在固定语义组下，设备数 D 主要改变的是调度竞争与共享路径等待项，而非单位记录空间斜率；相应地，时间与吞吐会随各模式原生路径组织方式发生阶段性振荡。

这一结果与第 4 章模型中的 $W_m(D, r)$ 完全对应：本地多设备阶段最先被放大的，不是编码体量本身，而是调度竞争和共享路径抖动。因此，D 的增大并不会自动把 binary 推向统一的“更优”或“更差”，而是会让不同 peer 原生路径的等待项被分别放大出来。

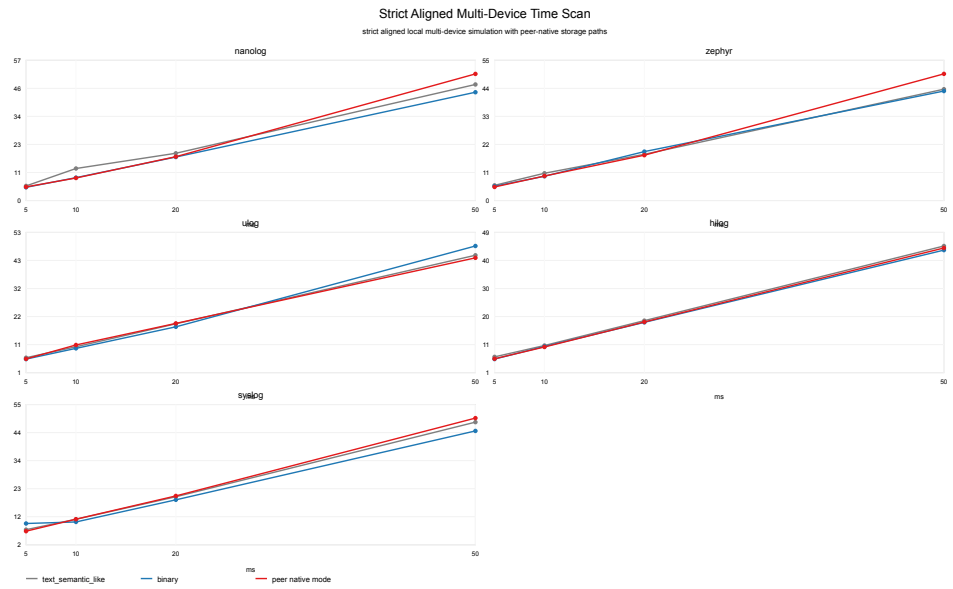

图 **5.5** 严格语义对齐本地多设备时间扫描

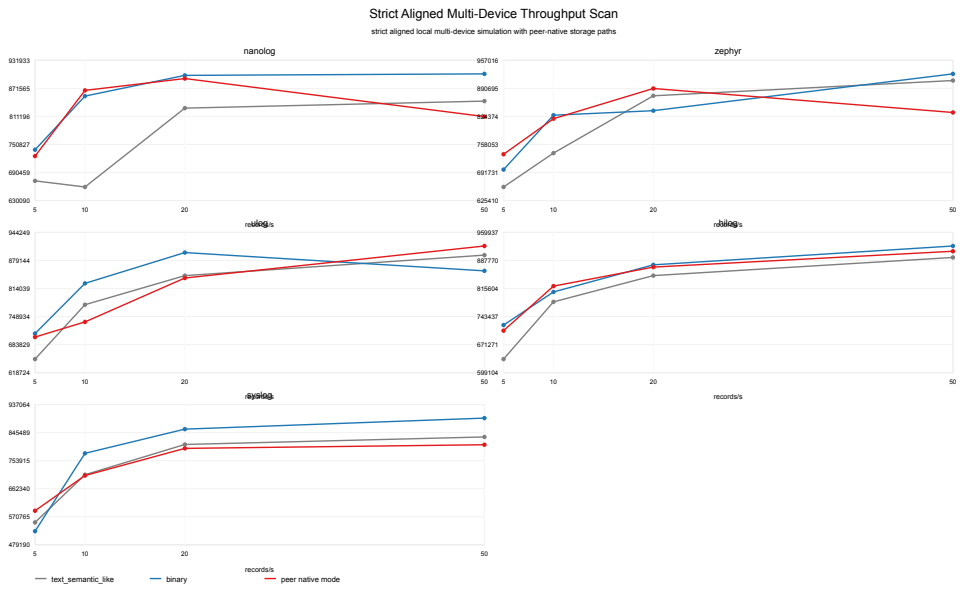

图 **5.6** 严格语义对齐本地多设备吞吐扫描

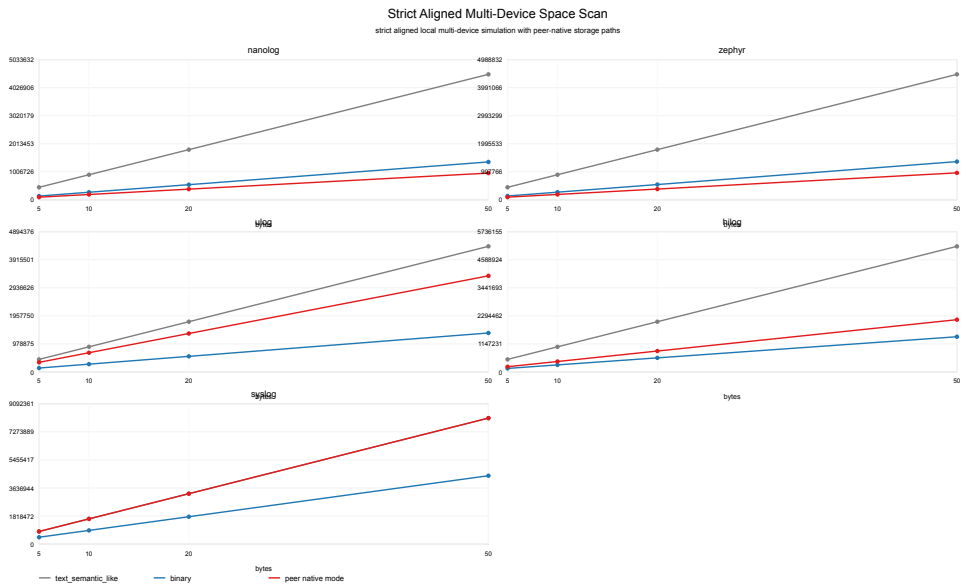

图 **5.7** 严格语义对齐本地多设备空间扫描

5.5 真实多设备：共享格式文件下的节点级结果

本节结果来自严格对齐套件的 11 节点结果及其合并汇总，采用 5/10/15/20 个并发节点、每节点一台设备的扫描方式。与本地多设备不同，这一场景在保持语义完全对齐的同时，引入了节点级冷启动、VM 调度噪声和跨节点执行偏移，因此更接近论文中的“工程部署代理场景”。本轮真实多设备复测统一使用硬件 CRC32C（aarch64 + -march=armv8-a+crc），并额外约束“全节点共享同一 shared_eventtag.bin 格式缓存文件”，用于验证共享元数据在节点级并发下的趋势变化。为便于与前文表格保持一致，表5.5先给出 20 节点最大规模的 binary vs text 截面，表5.6再给出 binary vs peer 的节点扫描结果。

表 5.5 真实多设备模拟下 binary 相对 text 的结果

分组	节点有效数	binary 相对 text 时间%	binary 相对 text 空间%	binary 相对 text 吞吐%
syslog	20/20	+61.26%	+45.76%	+140.07%
ftrace	20/20	+32.42%	+56.12%	+44.40%
nanolog	20/20	+33.61%	+69.81%	+51.02%
zephyr	20/20	+35.69%	+69.54%	+52.16%
ulog	20/20	+33.40%	+68.98%	+49.73%
hilog	20/20	+45.00%	+71.97%	+77.14%

表 5.6 真实多设备模拟下 binary 相对 peer 的结果

分组	节点数	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	现象
syslog	5	+62.49%	+45.76%	+129.90%	三指标同向为正
syslog	10	+58.26%	+45.76%	+122.45%	中节点规模稳定领先
syslog	15	+55.30%	+45.76%	+119.28%	节点提升后优势延续
syslog	20	+63.04%	+45.76%	+128.09%	20 节点仍为稳定正区
ftrace	5	-201.92%	+65.81%	-66.25%	空间为正，时间吞吐为负
ftrace	10	-217.27%	+65.81%	-67.80%	“空间优、时间劣”结构稳定
ftrace	15	-157.97%	+65.81%	-60.76%	结构方向保持一致
ftrace	20	-185.93%	+65.81%	-65.73%	20 节点结构未改变
nanolog	5	-203.24%	-41.68%	-67.09%	紧凑路径优势显著

（续表 5.6）

分组	节点数	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	现象
nanolog	10	-176.39%	-41.68%	-65.03%	负区保持但幅度有波动
nanolog	15	-196.70%	-41.67%	-67.38%	节点提升后再次回落
nanolog	20	-196.45%	-41.67%	-66.28%	20 节点仍未接近交叉
zephyr	5	-100.24%	-41.69%	-50.25%	deferred 路径保持占优
zephyr	10	-104.54%	-41.68%	-52.07%	阶段性波动但未转正
zephyr	15	-134.20%	-41.67%	-55.74%	节点增加后进一步回落
zephyr	20	-115.32%	-41.67%	-53.42%	20 节点仍为负区
ulog	5	+15.79%	+59.45%	+16.10%	低节点已整体转正
ulog	10	+12.10%	+59.45%	+17.60%	中节点优势稳定

（续表 5.6）

分组	节点数	binary 相对 peer 时间%	binary 相对 peer 空间%	binary 相对 peer 吞吐%	现象
ulog	15	+15.81%	+59.45%	+19.98%	节点增加后仍稳定为正
ulog	20	+14.47%	+59.45%	+15.31%	20 节点仍具综合优势
hilog	5	-22.95%	+32.65%	-18.65%	时间吞吐为负，空间为正
hilog	10	-11.56%	+32.66%	-12.58%	负区收敛但未转正
hilog	15	-18.73%	+32.66%	-14.04%	节点提升后再次回落
hilog	20	-13.02%	+32.66%	-12.36%	20 节点仍呈“空间优、时间吞吐负”

如表5.5和表5.6所示，真实多设备场景比本地多设备呈现出更稳定的分组结构。首先，在 20 节点规模下，binary vs text_semantic_like 在六个语义组均保持显著正收益，说明节点级并发条件下文本路径仍是高开销基线。其次，syslog/ulog 在 5/10/15/20 节点上均保持综合正收益。再次，nanolog/zephyr 在“共享格式文件只计一次”的口径下仍持续处于负区，ftrace 稳定表现为“空间正收益、时间与吞吐负收益”，hilog 则呈现“空间持续占优、时间与吞吐持续负”的结构。该结果说明，节点级并发与共享文件竞争会进一步放大不同 peer 原生路径的结构差异。

因此，“真实多设备下谁更优”并不是单机结论的简单复制，而是对第 4 章

模型中 $W_m(D,r)$ 与冷路径项 A_m 的联合验证：syslog/ulog 这类文本扩展或系统路径开销更重的 peer，在三指标上更容易被 binary 持续压过；hilog 在该轮节点级口径下表现为“空间更优但时间与吞吐劣势”；nanolog/zephyr 因原生路径足够紧凑，在该口径下仍未越过交叉点；ftrace 则呈现“空间更紧凑但时延更高”的 tracing 路径特征。

需要说明的是，本节表格里的空间指标统一采用“集群总空间且共享格式文档只计一次”口径，因此不会再出现“共享文档按节点重复累计”的偏差；在该口径下，空间绝对值仍随节点数近似线性增长，与第 4 章线性空间模型一致。

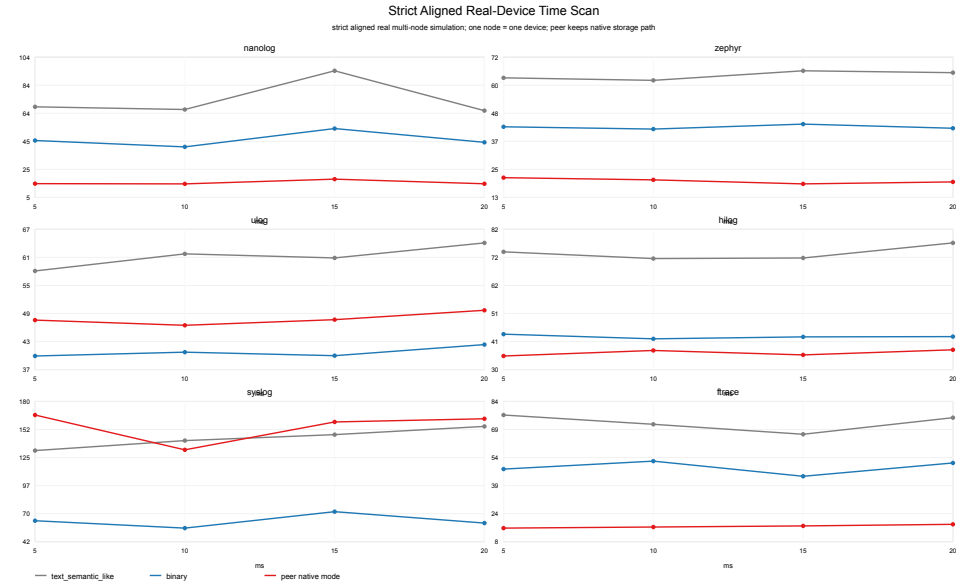

图 **5.8** 严格语义对齐真实多设备时间扫描

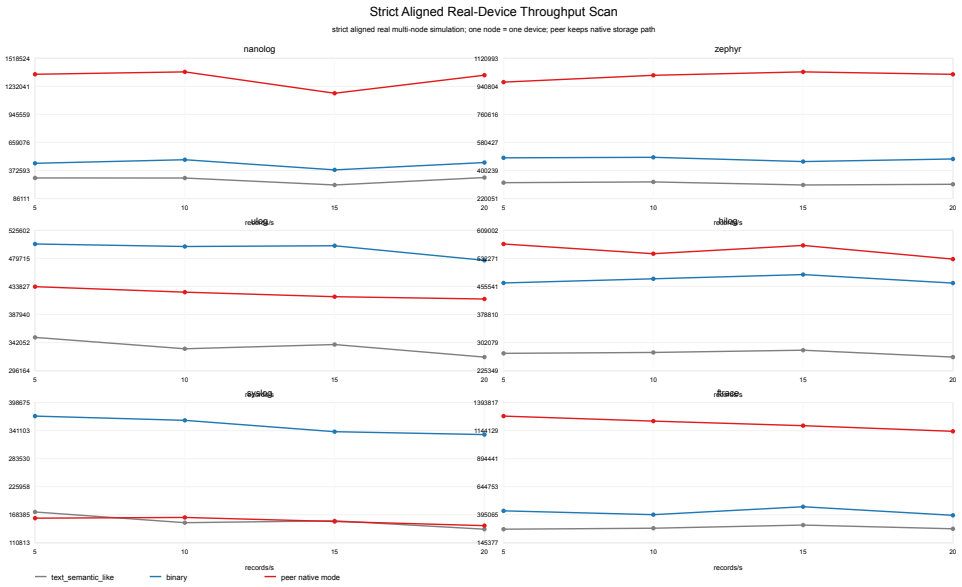

图 **5.9** 严格语义对齐真实多设备吞吐扫描

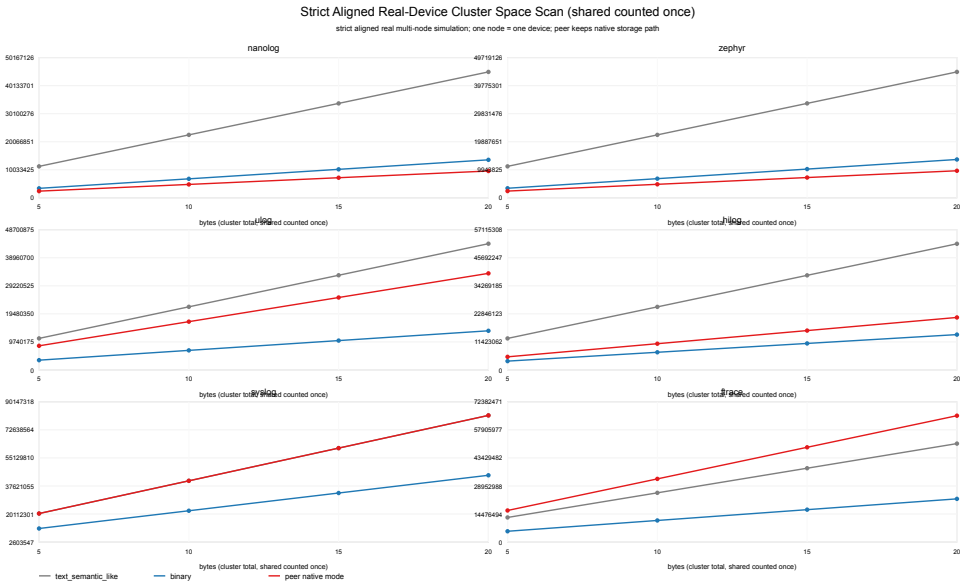

图 **5.10** 严格语义对齐真实多设备集群总空间扫描

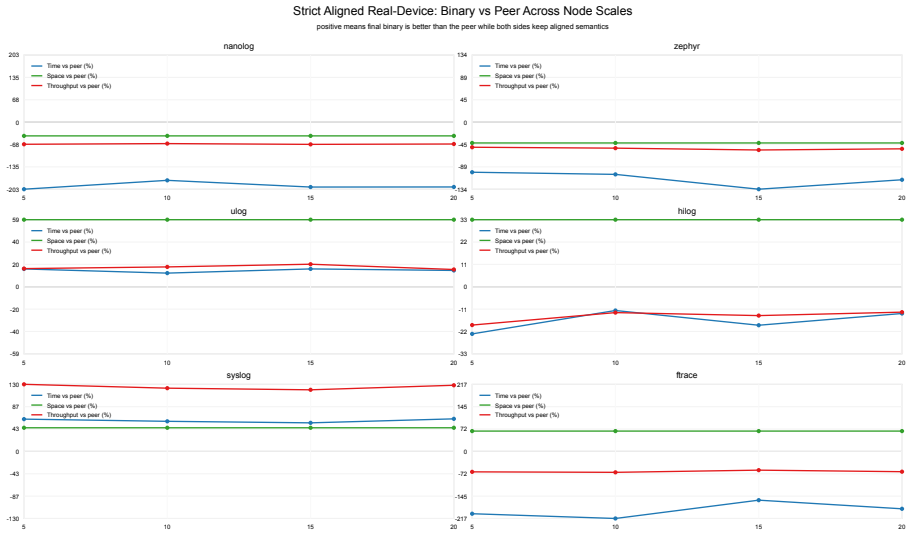

图 **5.11** 严格语义对齐真实多设备场景中 binary 相对 peer 的节点扫描差值

图5.8、图5.9 与图5.11 共同支持“syslog/ulog 在时间-吞吐维度整体为正、hilog 在该轮节点级结果中时间与吞吐均为负、nanolog/zephyr 持续负区、ftrace 呈空间正收益但时间吞吐负收益”的主结论；图5.10 则给出空间口径的直接证据：当统计对象改为“集群总空间”后，各组曲线均随节点数近似线性增长，与第 4 章线性空间模型一致。

5.6 日志量扩展：空间交叉点实验

为了回答“日志量增加到什么程度，Optbinlog 的空间性能会优于其他模式”这一问题，本文在严格语义对齐口径下新增 N 扫描实验：单设备单进程，模式固定为 text_semantic_like / binary / peer native mode，记录数扫描点为 1,2,5,10,20,50,100,200,500,1000,5000,20000,100000，每点 repeats=2、warmup=0，统计指标为 total_bytes 均值。交叉点定义为“最早满足 binary_bytes ≤ peer_bytes 的 N”。

表 5.7 日志数据量扫描下 binary 相对 peer 的空间交叉点结果

分组	经验交叉点 N^*	线性拟合交叉点	趋势解读
syslog	10	6.0	$N \approx 10$ 附近出现交叉，后续保持空间正收益
nanolog	未出现（至 100000）	-43.9	正 N 区间无交叉，binary 空间持续劣势
zephyr	未出现（至 100000）	-45.2	正 N 区间无交叉，binary 空间持续劣势
ulog	10	9.9	$N \approx 10$ 即出现交叉，后续保持空间正收益
hilog	50	29.7	$N \approx 30 \sim 50$ 出现交叉，小 N 阶段 peer 仍占优

如表5.7、图5.12与图5.13所示，空间维度呈现清晰分组。syslog/ulog 在较小日志量（约 $N = 10$）即越过交叉点，说明文本展开或系统路径带来的增量字节会较快累积；hilog 在 $N \approx 30 \sim 50$ 左右完成反超，表明其轻量头路径在小日志量下仍保留一定优势；nanolog/zephyr 两组在扫描上限 $N = 100000$ 仍未出现交叉，说明其 peer 原生路径的单位记录空间效率在当前 schema 下持续优于 binary。该实

验给出了可操作的工程判据：若目标场景对标 syslog/ulog 且日志量达到 $N \geq 10$，或对标 hilog 且日志量达到 $N \geq 50$，Optbinlog 在空间上可预期优于 peer；若对标 nanolog/zephyr，则应优先优化 binary 的单位记录编码开销，再讨论交叉点外推。

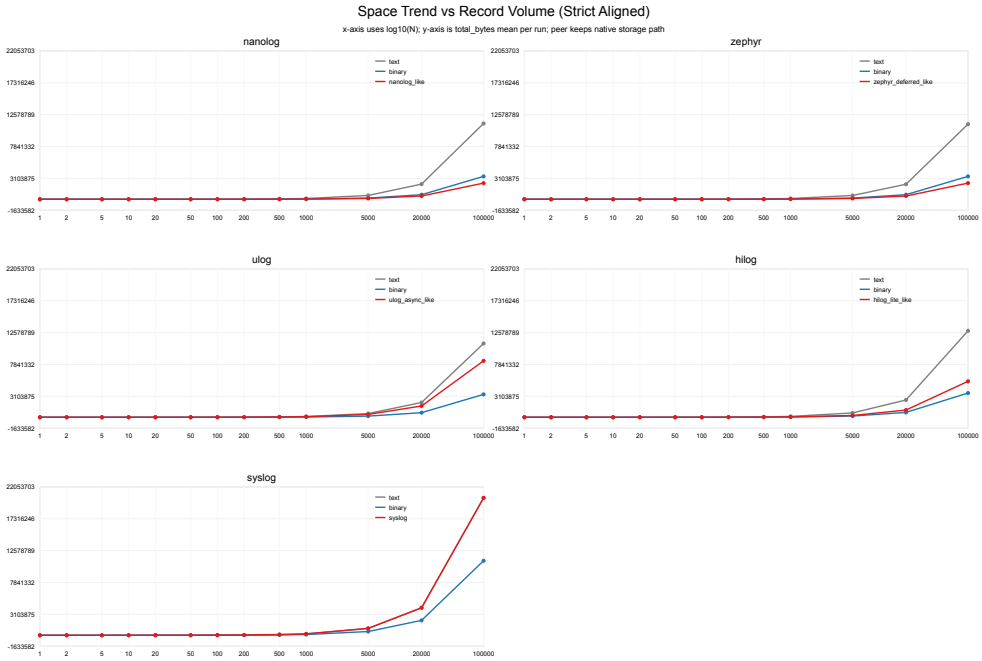

图 **5.12** 日志数据量扫描下各模式空间曲线

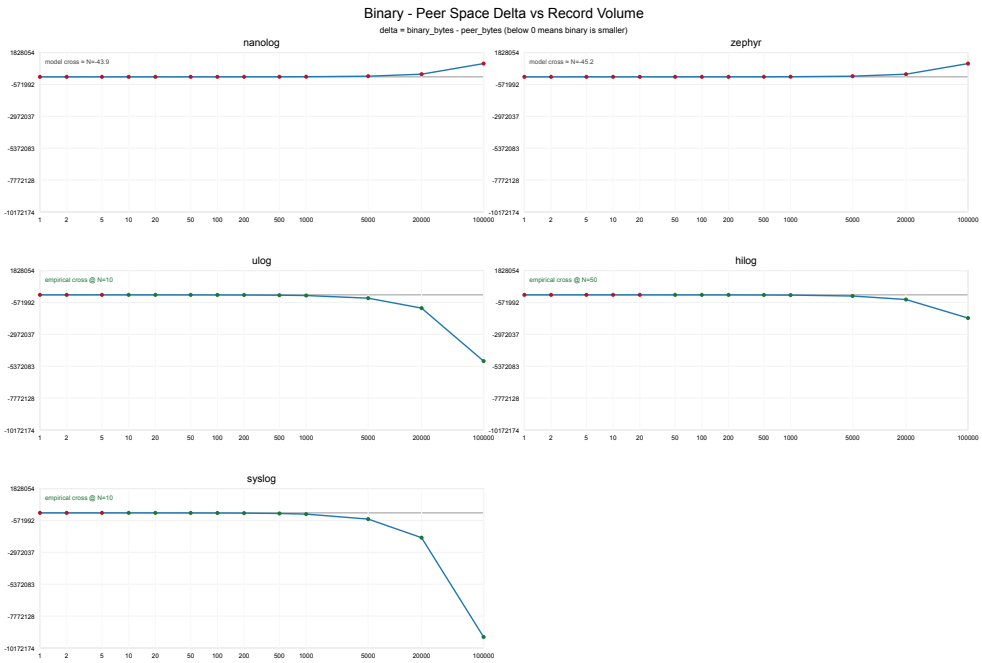

图 **5.13** 日志数据量扫描下 binary 相对 peer 的空间差值

5.7 理论—实证对照

综合 5.3 ~ 5.6 与第 4 章模型，可得到五点结论：

1. 空间模型与实测保持一致：total_bytes 随 N 近似线性增长，不同语义组对应不同斜率 B_m。nanolog/zephyr 组中 binary 空间持续为负，syslog/ulog/hilog 组中 binary 空间持续为正，l1 的 ftrace 组同样保持空间正收益。
2. 时间与吞吐交叉点具有 “语义组相关、平台相关” 特征，而非统一阈值。nanolog/zephyr 在 single 可转正，但在 l1 再次转负；syslog/ulog 在 l1 保持稳定正收益；hilog 在 l1 呈 “空间正、时间吞吐负” 的分化形态；ftrace 在 l1 时间维度持续负区。
3. 竞争项 $W_m(D,r)$ 在本地多设备与真实多设备中均可观测。本地多设备表现为 “时间/吞吐振荡、空间斜率稳定”，真实多设备则进一步放大为 “持续正区/持续负区/近持平” 的结构性分化。
4. 真实多设备不是本地多设备的线性外推。l1 场景引入节点冷启动、VM 调度噪声与进程重建成本，使固定项 A_m 占比上升，并直接改变不同模式的收益分布。
5. 日志量扩展实验给出可操作阈值：syslog/ulog 在 $N \approx 10$ 左右完成空间反超，hilog 反超点约在 $N \approx 30 \sim 50$，nanolog/zephyr 在 $N \leq 100000$ 仍未出现空间交叉。

5.8 原因分析与消融佐证

为了更直接解释上述三层场景的排序差异，本文继续使用严格公平消融运行作为 “原因分析层”，但不再把它作为主体结论数据集。该数据集给出的关键消融结果如下：

表 5.8 消融实验中的冷路径与校验影响

分组	binary 冷路径 prep(ms)	binary_nocrc 时间改善%
syslog	8.214	+2.45%
nanolog	0.680	-2.82%
zephyr	0.703	+22.60%
ulog	0.713	+4.59%
hilog	0.827	+0.04%

表 5.9 消融实验中的字符串编码影响

分组	binary_varstr 时间改善%	binary_nocrc_varstr 时间改善%
syslog	-2.41%	+3.89%
nanolog	-6.86%	+0.38%
zephyr	+4.36%	+16.09%
ulog	-8.85%	-4.41%
hilog	+3.63%	+3.58%

如表5.8和表5.9所示，结合严格对齐结果与消融结果，可以把原因归纳为三点：1. **校验开销并非统一主导项**：本地五组中，binary_nocrc 的影响区间扩大到 -2.82% ~ +22.60%。这说明关闭校验并不会在所有语义组上自动带来收益，真正受益更明显的是 zephyr 这类紧凑定长路径。2. **冷路径差异仍然存在，但不再只集中于文本型 peer**：syslog 的 prep_ms 仍达到 8.214ms，说明系统路径初始化仍然最重；而 nanolog/zephyr/ulog/hilog 的 prep_ms 已落在 0.680 ~ 0.827ms，说明单机主导差异更多开始转向热路径组织方式，而不是单纯的初始化成本。3. **auto-varstr 收益具有明显语义组差异**：zephyr/hilog 中 varstr 或 nocrc_varstr 仍可能带来正收益，但 nanolog/ulog/syslog 中单独开启 varstr 反而可能转负，说明变长编码并非所有 schema 的统一最优策略，必须结合字段分布来判断。

5.9 证据链闭环与外推边界

本文主证据链由“最新严格公平层 + 初始化竞争验证 + 空间交叉点扫描”组成：1. **严格公平层主数据**：text_semantic_like / binary / peer native mode 在 single、本地多设备与真实多设备三层结果。2. **正确性验证数据**：本地与节点级初始化竞争，验证“单创建者 + 多复用者”并发语义。3. **阈值验证数据**：$N = 1 \sim 100000$ 空间扫描，验证不同语义组的空间交叉点是否存在。

该证据链使结论具备三重可检验性：指标可复核、机制可解释、阈值可复现。工程近似层在本章中仅用于补充说明，不承担最终排序结论。

5.10 机制解释与应用启示

结合最新严格公平结果与消融数据，可将 Optbinlog 的分组结论概括为：1. **稳定优势组（syslog/ulog）**：在 single 与真实多设备均保持清晰正收益，是当前

最确定的应用受益区间。2. **中间过渡组（hilog）**：空间长期占优，但在节点级并发下时间与吞吐转为负区，体现“空间稳、时间敏感”的过渡态。3. **高压竞争组（nanolog/zephyr）与 tracing 组（ftrace）**：前者在当前口径下仍未跨越节点级交叉点，后者维持“空间正收益、时间吞吐负收益”。

由此可见，Optbinlog 的工程价值主要体现在三点：跨进程一致性强、语义扩展成本低、记录级可校验；其后续优化重点应继续聚焦于冷路径压缩、稳态缓存收益释放与批量提交效率提升。

---

6 结论与未来展望

6.1 研究结论

基于最新一版严格语义对齐数据以及初始化竞争、空间交叉点等辅助证据，本文得到以下结论：

1. binary 相对 text 在全部语义组上保持稳定正收益，说明结构化二进制路径在高频写入场景下具备明确基线优势。
2. 在 peer 原生路径对比中，syslog/ulog 属于稳定优势组，hilog 呈现“空间优势稳定、时间吞吐受节点级并发抑制”的过渡特征，nanolog/zephyr 在当前节点级口径下仍未完成反超，ftrace 保持“空间正收益、时间吞吐负收益”。
3. 空间交叉点具有语义组相关性：syslog/ulog 在 $N \approx 10$ 出现反超，hilog 在 $N \approx 50$ 左右出现反超，nanolog/zephyr 在 $N \leq 100000$ 仍未出现交叉。
4. 初始化竞争结果持续满足“单创建者 + 多复用者”约束，说明共享格式文件方案在本地与节点级并发下均保持正确性。

总体而言，本文提出的理论模型能够较好解释最新实验趋势，说明 Optbinlog 更适用于长期在线、强调跨进程一致性、结构化语义和结果可追溯性的嵌入式网关与边缘控制场景。

6.2 当前局限

1. 受网络与平台限制，尚未完成外部系统官方工程的全栈编译复现（当前为代码级阅读 + 算法级原型复现）。

2. Linux 实验仍以 VM 为主，尚未覆盖 Raspberry Pi / i.MX 实机。
3. 严格公平语义套件已经扩展到 20 节点 VM 级节点代理场景，但仍未迁移到真实 ARM 板卡或 bare-metal Linux 实机。
4. 当前严格公平套件采用“新进程 + 单次写入”的统计口径，因此新加入的进程内 shared/tag cache 收益尚未在长生命周期 steady-state 基准中得到充分量化。
5. 缺少 perf 级 CPU 微观指标，暂不能细粒度解释所有异常点。

6.3 未来展望

面向后续研究，本文认为可以从以下几个方向继续推进：1. 在更广泛的平台与负载条件下开展验证，进一步提升结论的外部有效性与可迁移性。2. 在保持语义一致性的前提下继续优化热路径与稳态路径，缩小与极短路径 peer 在特定场景下的差距。3. 完善日志可靠性与可恢复性评估，形成覆盖损坏、截断与异常恢复过程的系统化证据。4. 结合更细粒度观测手段（如 CPU 与调度行为分析），建立从机制到性能现象的更强解释闭环。

参考文献

[1] Linux Kernel Documentation. Lockless Ring Buffer Design[EB/OL]. https://www.kernel.org/doc/html/latest/trace/ring-buffer-design.html.

[2] Linux Kernel Documentation. user_events[EB/OL]. https://www.kernel.org/doc/html/latest/trace/user_events.html.

[3] LTTng Project. LTTng Documentation: Concepts[EB/OL]. https://lttng.org/docs/#doc-concepts.

[4] Zephyr Project. Logging[EB/OL]. https://docs.zephyrproject.org/latest/services/logging/index.html.

[5] Zephyr Project. Kconfig.mode[EB/OL]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/upstream/collab-sdk-dev/subsys/logging/Kconfig.mode.

[6] Zephyr Project. log_msg.c[EB/OL]. https://pigweed.googlesource.com/third_party/github/zephyrproject-rtos/zephyr/+/refs/heads/main/subsys/logging/log_msg.c.

[7] systemd. Journal File Format[EB/OL]. https://systemd.io/JOURNAL_FILE_FORMAT/.

[8] Yang S, Park W, Byan T, et al. NanoLog: A Nanosecond Scale Logging System[C]. USENIX Annual Technical Conference, 2018.

[9] PlatformLab. NanoLogCpp17.h[EB/OL]. https://raw.githubusercontent.com/PlatformLab/NanoLog/master/runtime/NanoLogCpp17.h.

[10] PlatformLab. RuntimeLogger.cc[EB/OL]. https://raw.githubusercontent.com/PlatformLab/NanoLog/master/runtime/RuntimeLogger.cc.

[11] Android Open Source Project. event.logtags[EB/OL]. https://android.googlesource.com/platform/system/logging/+/refs/tags/android-13.0.0_r33/logcat/event.logtags.

[12] RT-Thread. ulog 文档 [EB/OL]. https://rt-thread.github.io/rt-thread/page_component_ulog.html.

[13] RT-Thread. ulog.c[EB/OL]. https://raw.githubusercontent.com/RT-Thread/rt-thread/master/components/utilities/ulog/ulog.c.

[14] OpenHarmony. HiLog Lite README[EB/OL]. https://gitee.com/openharmony/

hiviewdfx_hilog_lite/blob/master/README_zh.md.

[15] Linux Kernel. ring_buffer.c[EB/OL]. https://raw.githubusercontent.com/torvalds/linux/master/kernel/trace/ring_buffer.c.

[16] systemd. journald-server.c[EB/OL]. https://github.com/systemd/systemd/blob/main/src/journal/journald-server.c.

[17] Sigelman B H, Barroso L A, Burrows M, et al. Dapper, a Large-Scale Distributed Systems Tracing Infrastructure[C]. Google Research, 2010.

[18] Cantrill B M, Shapiro M W, Leventhal A H. Dynamic Instrumentation of Production Systems[C]. USENIX Annual Technical Conference, 2004.

[19] Zhu J, He P, Fu Q, et al. Learning to Log: Helping Developers Make Informed Logging Decisions[C]. ICSE, 2015.

[20] OpenTelemetry. Logs Data Model[EB/OL]. https://opentelemetry.io/docs/specs/otel/logs/data-model/.

[21] Chen T, Song Q, Zhu T, et al. Kellect: A Kernel-based efficient and lossless event log collector for windows security[J]. Computers & Security, 2025, 150:104203.

[22] Balakrishnan R, Sahoo R K. Lossless compression for large scale cluster logs[C]. IPDPS, 2006.

[23] Binlog. Internals[EB/OL]. https://binlog.org/Internals.html.

致谢

本论文（设计）完成过程中，得到了指导教师在选题、方法、实验组织和写作规范方面的持续指导；也得到了课题组同学在实验环境、脚本复核与结果讨论方面的帮助。在此一并致以诚挚感谢。感谢学院和学校提供的学习与实验条件，感谢家人长期以来的理解与支持。

附录

A 关键复现实验命令（阶段化复现）

1. 语义对齐矩阵（single/multi/真实多设备模拟）

```
cd demo
python3 scripts/run_grouped_semantic_matrix.py
–out-dir ./results/paper_regen/grouped_semantic_matrix
```

2. 最终严格对齐套件（single/本地多设备/真实多设备）

```
python3 scripts/run_final_aligned_suite.py
–out-dir ./results/paper_regen/final_aligned_suite
```

3. 本地初始化竞争（10 进程）

```
mkdir -p build/bin
clang -O2 -Wall -Wextra -std=c11 -Iinclude
-o build/bin/optbinlog_init_race optbinlog_init_race.c
src/optbinlog_shared.c src/optbinlog_eventlog.c
OPTBINLOG_INIT_OUT_DIR=./results/paper_regen/init_race_local
OPTBINLOG_EVENTLOG_DIR=./eventlogst
OPTBINLOG_INIT_PROCS=10
OPTBINLOG_INIT_REPEATS=20
OPTBINLOG_INIT_WARMUP=3
python3 scripts/run_init_race.py
```

4. 节点级初始化竞争（每节点一台设备）

```
CFG=configs/l1_config.linux_10_all_unaligned_initrace.json
python3 scripts/run_l1_init_compete.py
–config configs/l1_config.linux_10_all_unaligned_initrace.json
–tag paper_init_compete
```

5. 便捷读取验证（字段名 + 统计）

```
mkdir -p build/bin
clang -O2 -Wall -Wextra -std=c11 -Iinclude
-o build/bin/optbinlog_read optbinlog_read.c
```

```
src/optbinlog_shared.c src/optbinlog_eventlog.c src/optbinlog_binlog.c
SAMPLE_DIR=./results/paper_dataset/samples/optbinlog_read
./build/bin/optbinlog_read
–shared ./results/paper_dataset/samples/optbinlog_read/shared_eventtag.bin
–log ./results/paper_dataset/samples/optbinlog_read/binary_run_000.bin
–format table –limit 20 –summary
```